

Organización de Computadoras

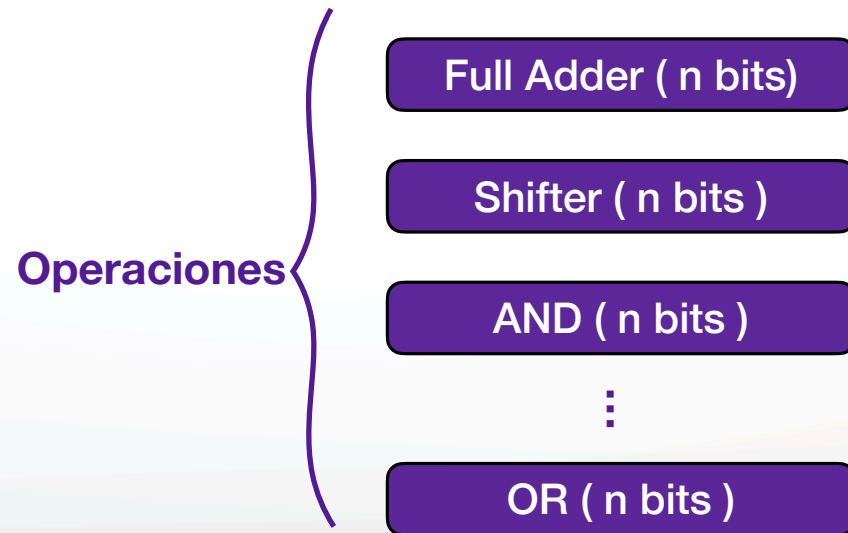
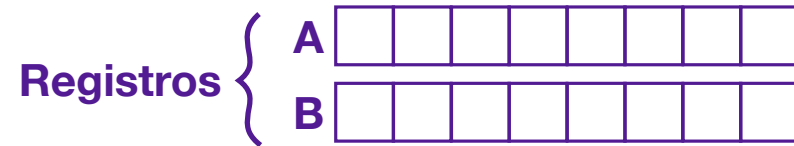
**Cómo computar utilizando la electricidad
Hacia una computadora de Propósitos Generales**

Departamento de Computación - UNRC

El camino a recorrer

- Un poco de Historia y Sistemas Numéricos
- Introducción a la Electrónica
- Representación de Información
- **Cómo computar utilizando la electricidad**
- Evolución y Funcionamiento abstracto de una computadora
- Assembly X86
- Micro-programación (cómo fabricar un procesador)
- Eficiencia
 - Pipelines
 - Memoria Caché
 - Memoria Virtual

Qué tenemos hasta ahora



Automatizar una sumatoria de n números

¿ Dónde guardamos
“la tarea”?

17

4

23

9

3

46

13

18

64

11

Automatizar una sumatoria de n números

CADA CELDA ALMACENA 8 BITS = 1 BYTE

¿ Dónde guardamos
“la tarea”?

17
4
23
9
3
46
13
18
64
11

MEMORIA

0	0	0	1	0	0	0	1	0
0	0	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	0	0	0	1	0	0	1	3
0	0	0	0	0	0	1	1	4
0	0	1	0	1	1	1	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	1	0	0	0	0	0	0	8
0	0	0	0	1	0	1	1	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

Tamaño de
memoria, cada
celda se
referencia con
una **dirección de
memoria**.
Por ejemplo,
16Bytes, de la
dirección
0 a la **F**

Automatizar una sumatoria de n números

CADA CELDA ALMACENA 8 BITS = 1 BYTE

Dirección de memoria

¿ Dónde guardamos
"la tarea"?

17

4

23

9

3

46

13

18

64

11

MEMORIA

0	0	0	1	0	0	0	1	0
0	0	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	1
0	0	0	0	1	0	0	1	1
0	0	0	0	0	0	1	1	1
0	0	1	0	1	1	1	0	1
0	0	0	0	1	1	0	1	1
0	0	0	1	0	0	1	0	1
0	1	0	0	0	0	0	0	0
0	0	0	0	1	0	1	1	1
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

0

1

2

3

4

5

6

7

8

9

A

B

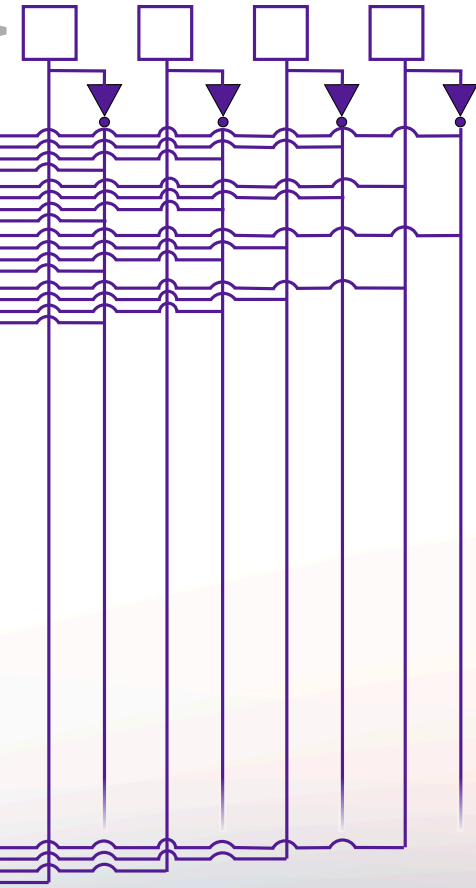
C

D

E

F

Tamaño de memoria, cada celda se referencia con una **dirección de memoria**. Por ejemplo, **16Bytes**, de la dirección **0** a la **F**



Automatizar una sumatoria de n números

CADA CELDA ALMACENA 8 BITS = 1 BYTE

¿ Dónde guardamos
“la tarea”?

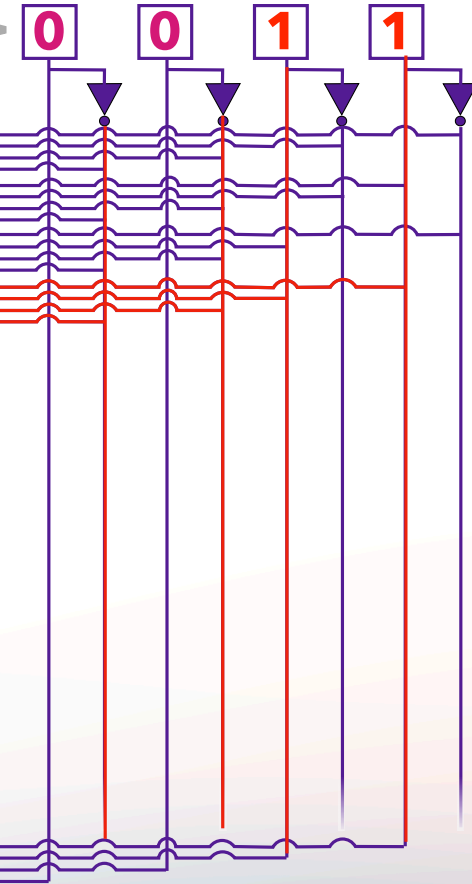
17
4
23
9
3
46
13
18
64
11

MEMORIA

0 0 0 1 0 0 0 1	0
0 0 0 0 0 1 0 0	1
0 0 0 1 0 1 1 1	2
0 0 0 0 1 0 0 1	3
0 0 0 0 0 0 1 1	4
0 0 1 0 1 1 1 0	5
0 0 0 0 1 1 0 1	6
0 0 0 1 0 0 1 0	7
0 1 0 0 0 0 0 0	8
0 0 0 0 1 0 1 1	9
0 0 0 0 0 0 0 0	A
0 0 0 0 0 0 0 0	B
0 0 0 0 0 0 0 0	C
0 0 0 0 0 0 0 0	D
0 0 0 0 0 0 0 0	E
0 0 0 0 0 0 0 0	F

0 0 0 0 1 0 0 1

Dirección de memoria



Tamaño de memoria, cada celda se referencia con una **dirección de memoria**. Por ejemplo, **16Bytes**, de la dirección **0** a la **F**

Automatizar una sumatoria de n números

CADA CELDA ALMACENA 8 BITS = 1 BYTE

Dirección de memoria

¿ Dónde guardamos
“la tarea”?

17

4

23

9

3

46

13

18

64

11

MEMORIA

0	0	0	1	0	0	0	1	0
0	0	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	1
0	0	0	0	1	0	0	1	1
0	0	0	0	0	0	1	1	1
0	0	1	0	1	1	1	0	1
0	0	0	0	1	1	0	1	1
0	0	0	1	0	0	1	0	1
0	1	0	0	0	0	0	0	0
0	0	0	0	1	0	1	1	1
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

0 0 0 0 1 0 0 1

Tamaño de
memoria, cada
celda se
referencia con
una **dirección de
memoria**.
Por ejemplo,
16Bytes, de la
dirección
0 a la **F**

■ decodificador (4 a 16)

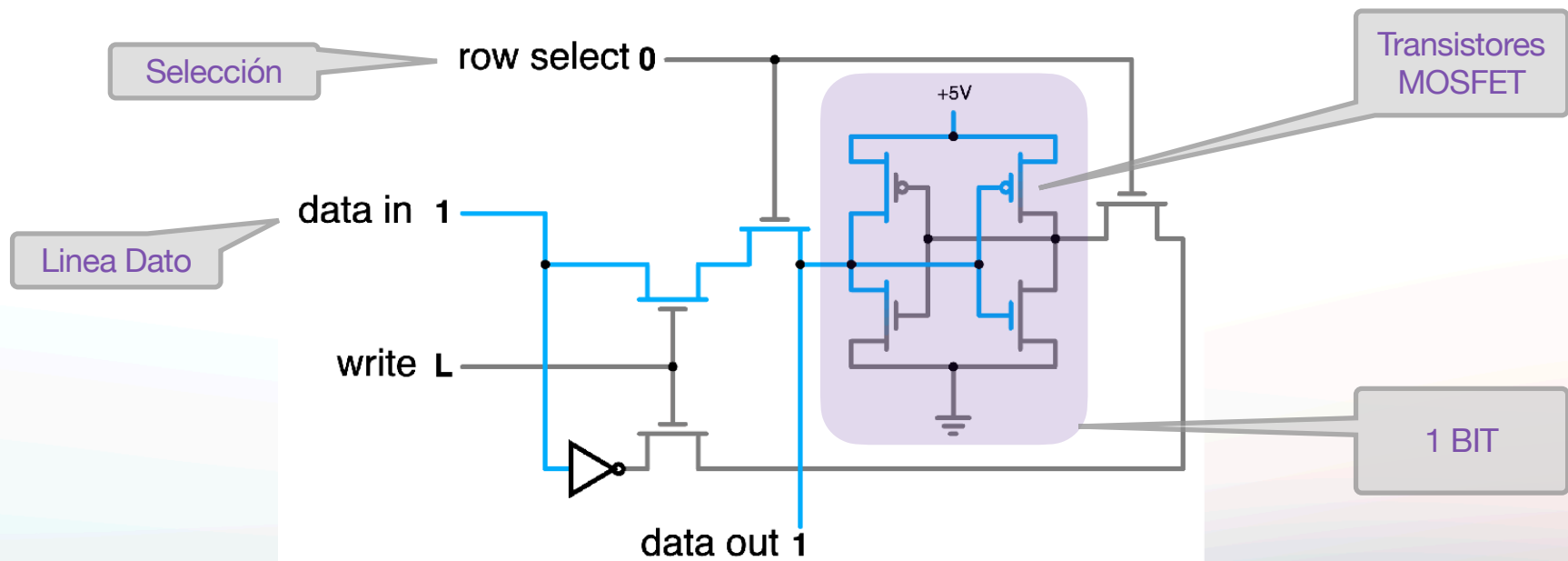
0 0 1 1

Tipos de Memorias

- **RAM (Memoria de Acceso Directo):** Comunmente denominada memoria principal. Es la memoria (lectura/escritura) con la que trabajamos cuando programamos, en la cual residen los programa y datos. Es **volátil** (sus datos solo se mantiene mientras tienen energía eléctrica). Si bien existe muchas variantes, podemos clasificarlas según su composición física en **estáticas** SRAM y **dinámicas** DRAM.
- **ROM (Memoria de sólo Lectura):** A diferencia de las RAM, sus datos **persisten aún sin energía eléctrica**. En genera se utilizan en pequeñas unidades conteniendo información (programas) indispensables para el funcionamiento, como el arranque, configuración de dispositivos, software embebido, etc. Dependiendo de su conformación y modo de grabado existen PROM, EPROM, EEPROM ...

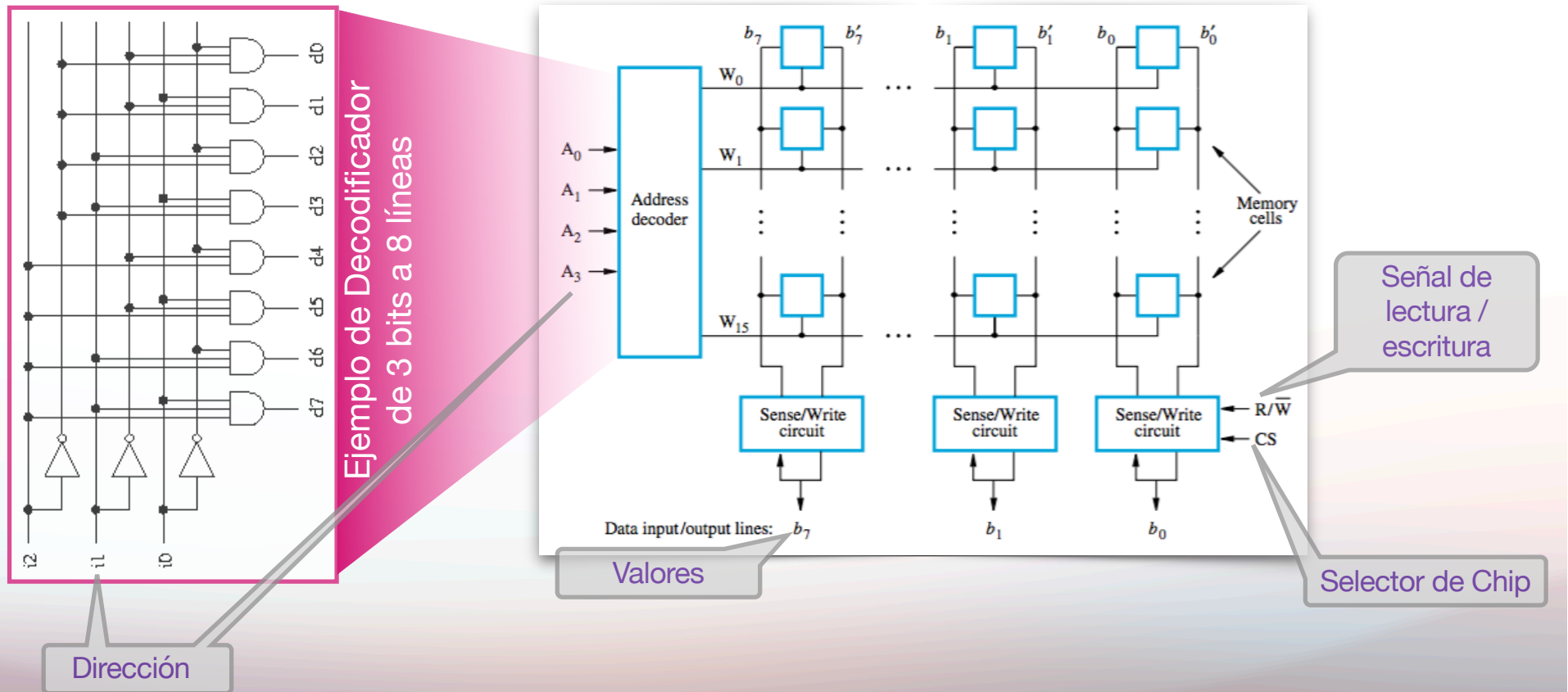
Memorias SRAM

- **SRAM (Static RAM)**: Está compuesta por circuitos similares a los Flip-Flop. Son memorias de acceso rápido pero su construcción es costosa. Consumen más energía y por lo tanto disipan más calor.



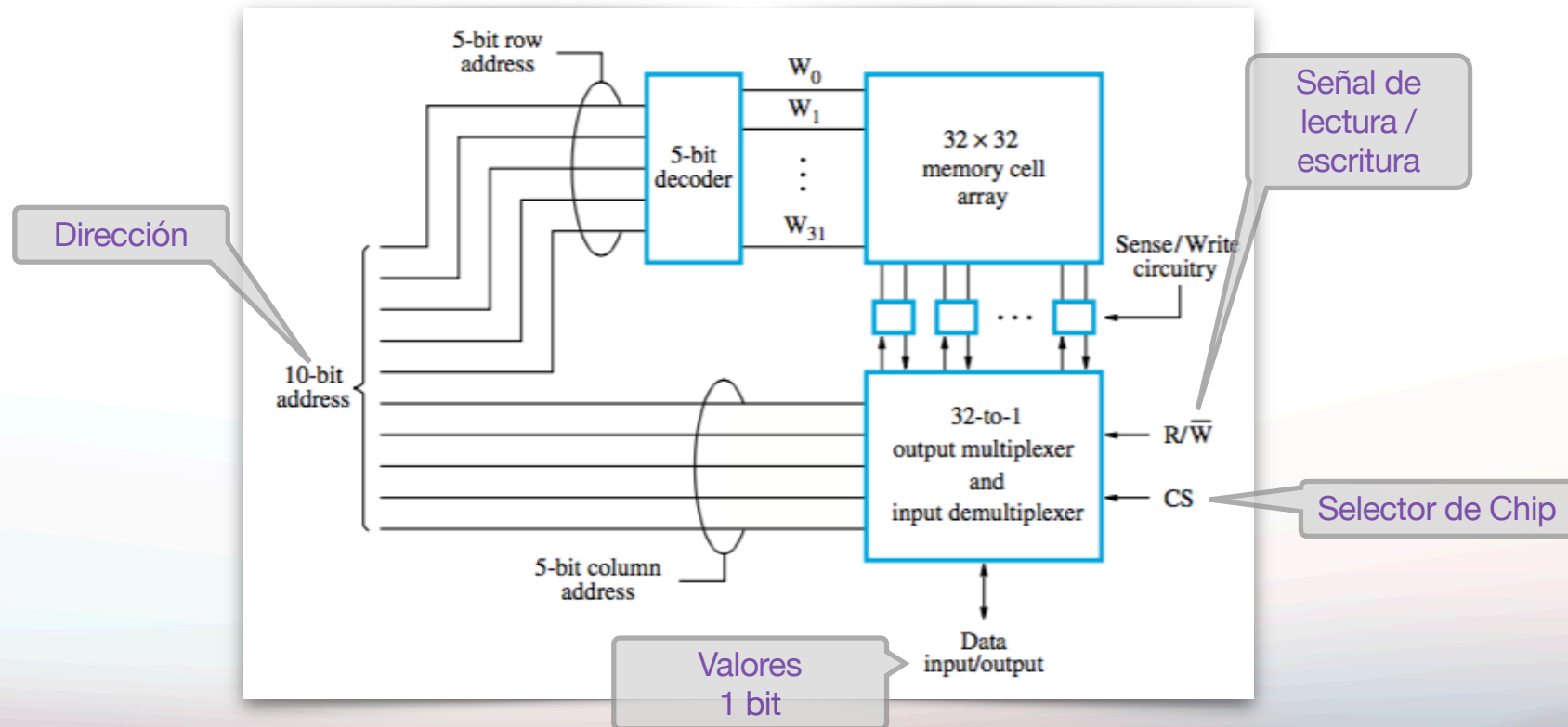
Memorias SRAM

- **SRAM (Static RAM):** Ejemplo de una organización 128B x 8



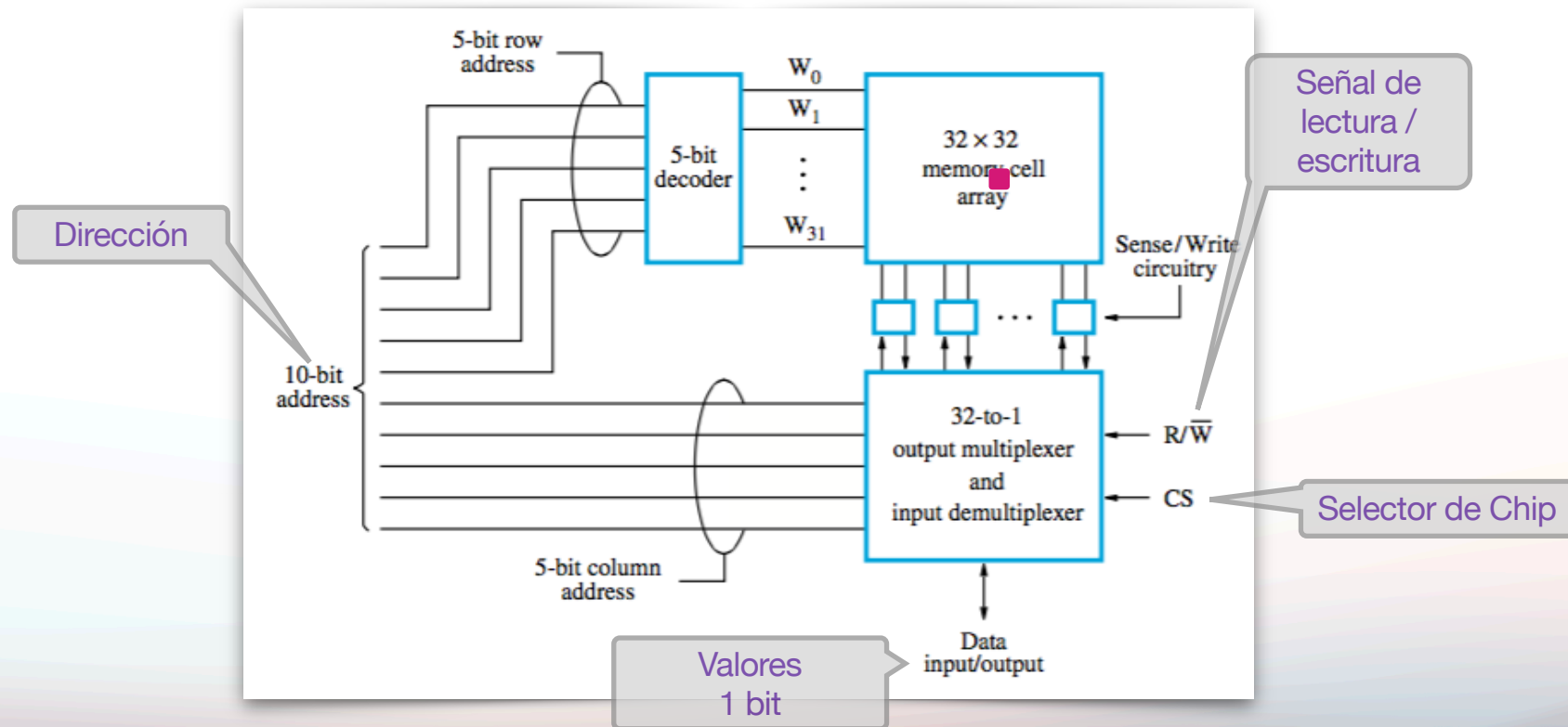
Memorias SRAM

- **SRAM (Static RAM):** Ejemplo de una organización 1024 (1K) x 1



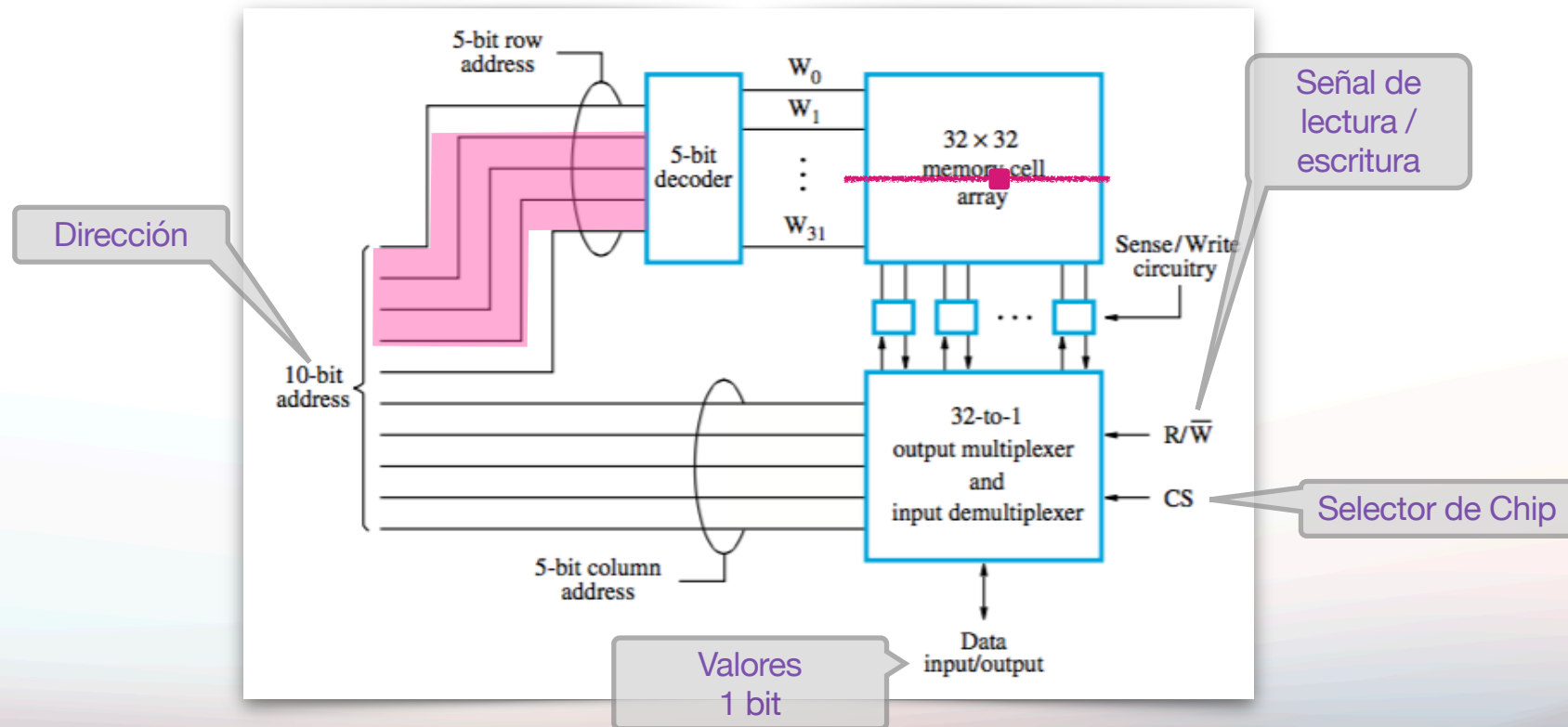
Memorias SRAM

- **SRAM (Static RAM)**: Ejemplo de una organización 1024 (1K) x 1



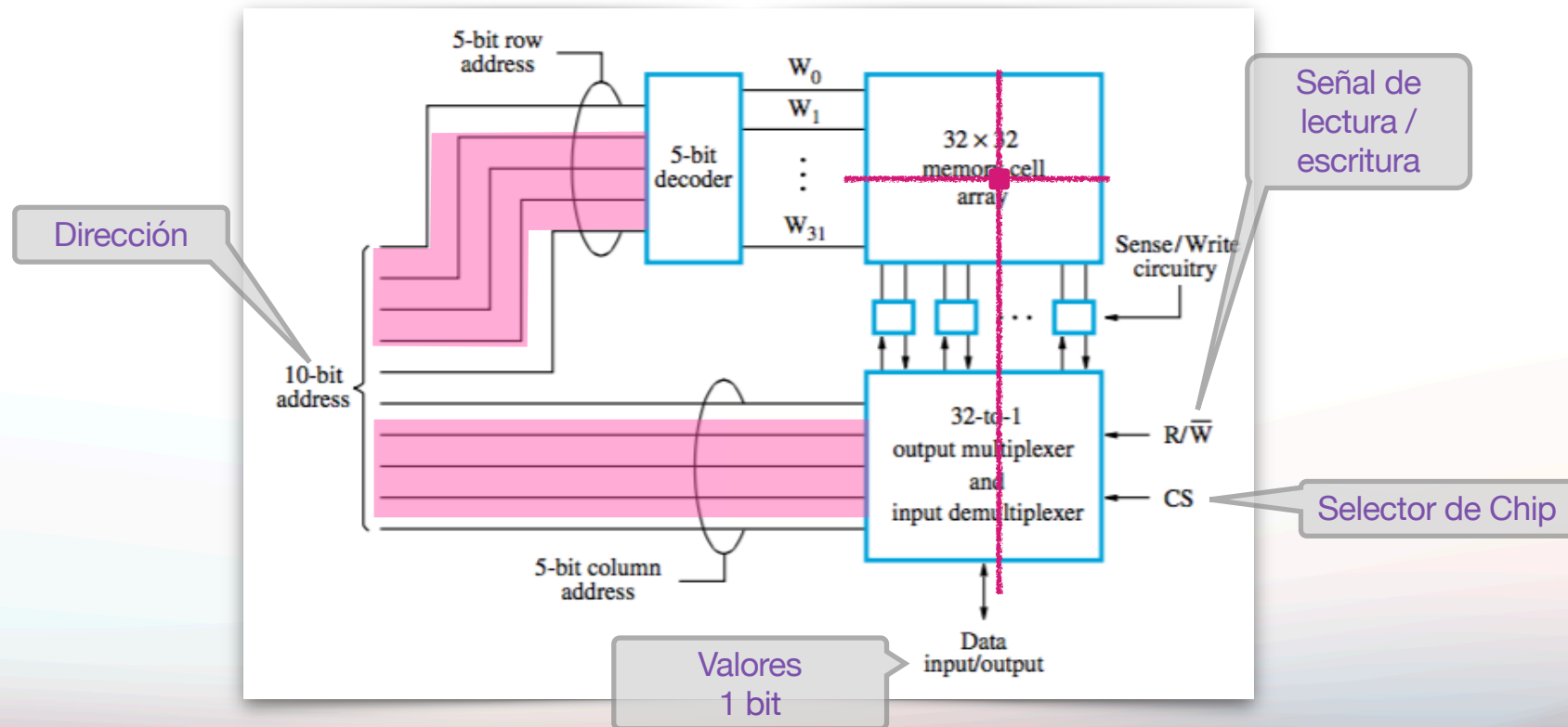
Memorias SRAM

- **SRAM (Static RAM):** Ejemplo de una organización 1024 (1K) x 1



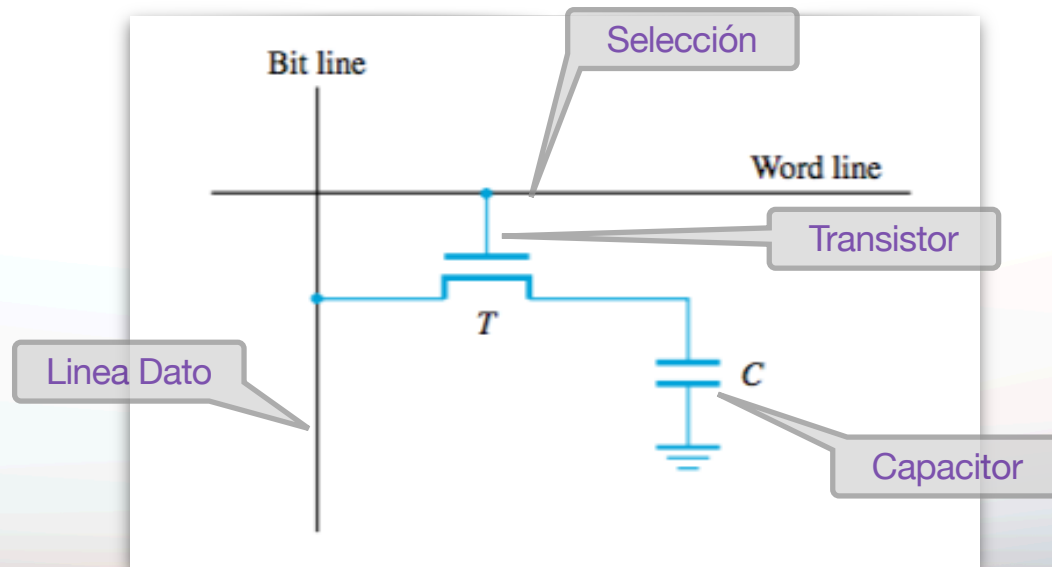
Memorias SRAM

- **SRAM (Static RAM):** Ejemplo de una organización 1024 (1K) x 1



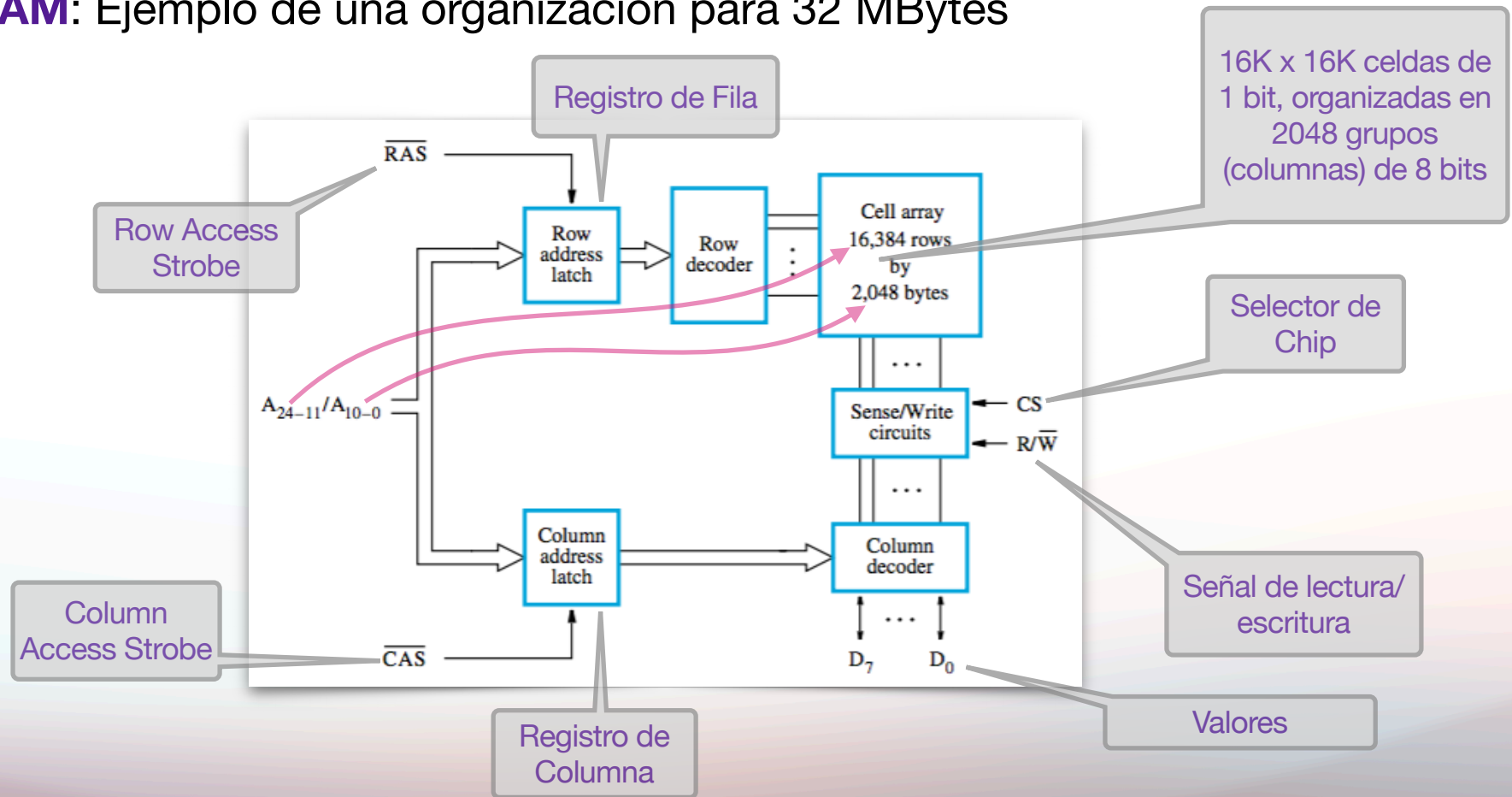
Memorias DRAM

- **DRAM (Dynamic RAM)**: Las celdas de memoria dinámica utilizan capacitores en lugar de transistores, son más baratas, consumen menos energía. Otra diferencia es que su valor debe ser revalidado (refresh).



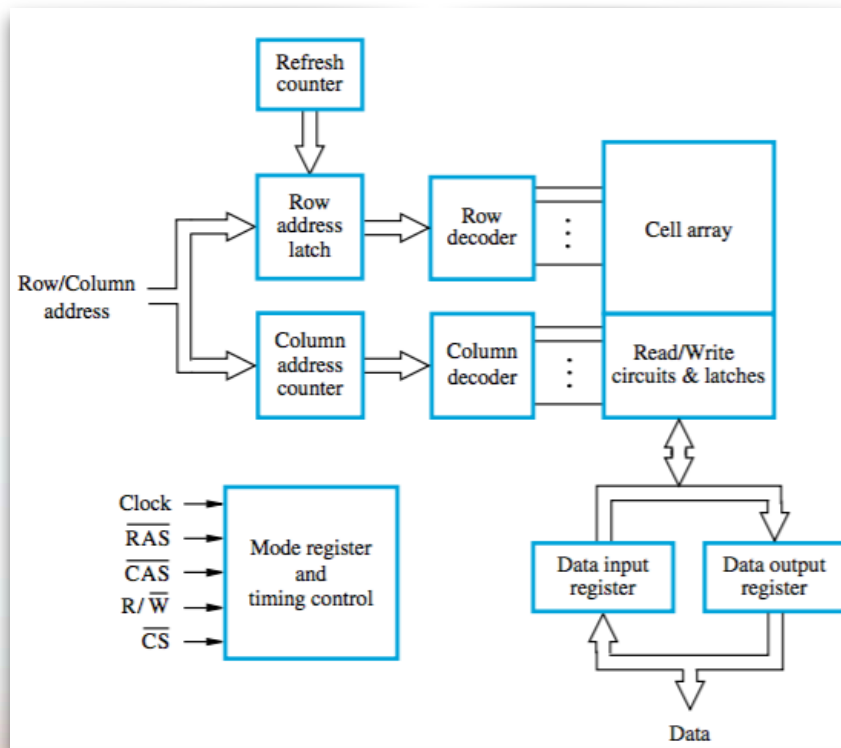
Memorias DRAM

- **DRAM**: Ejemplo de una organización para 32 MBytes



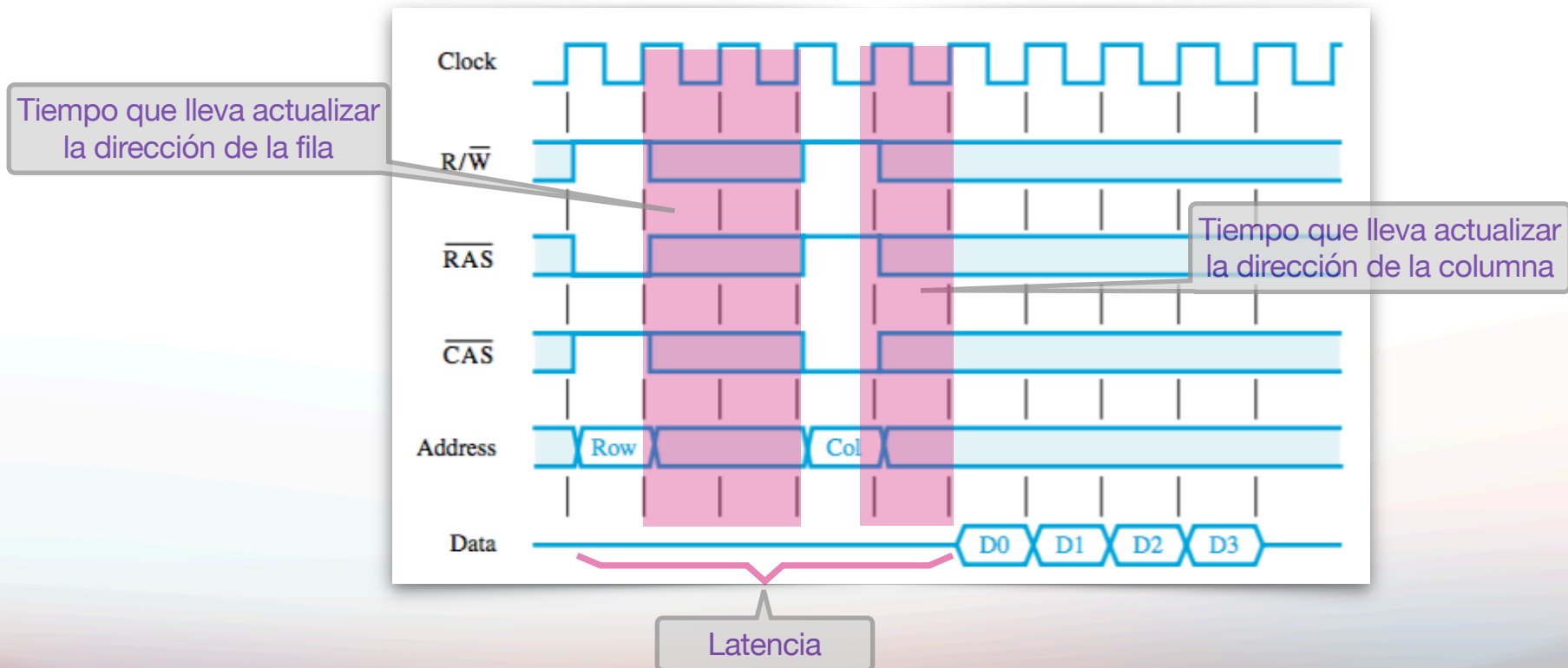
Memorias SDRAM

- **SDRAM (Synchronous Dynamic RAM):** A diferencia de las DRAM, la memoria SDRAM está sincronizada mediante un clock. Esto le otorga mayor control dentro del chip.



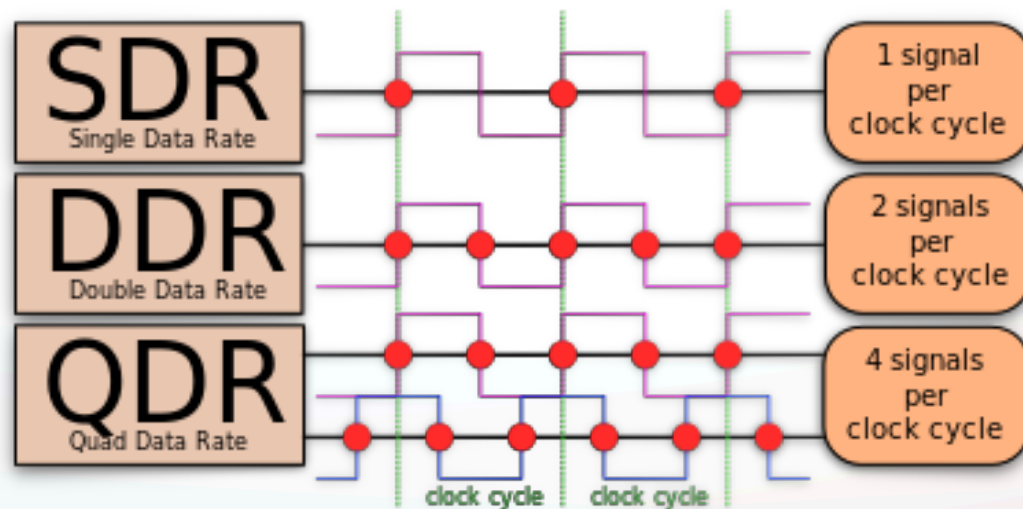
Memorias SDRAM

- **SDRAM (Synchronous Dynamic RAM):** Tiempos de acceso y latencia



Memorias SDRAM

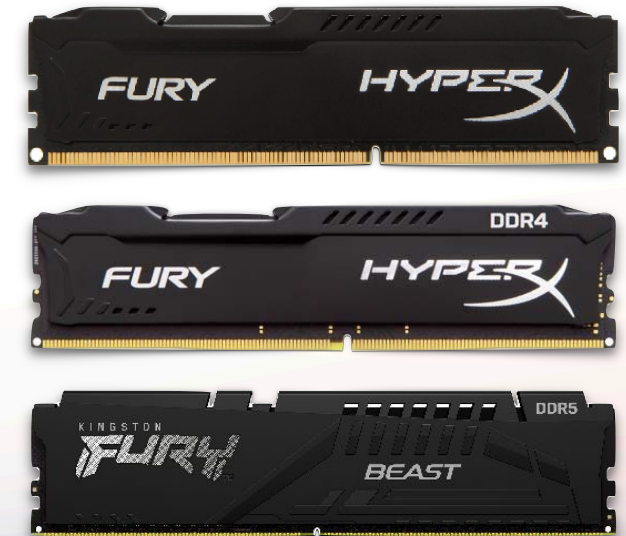
- **DDR (Double Data Rate):** Técnica de acceso para mejorar la eficiencia



Memoria principal - Latencia y Bandwidth

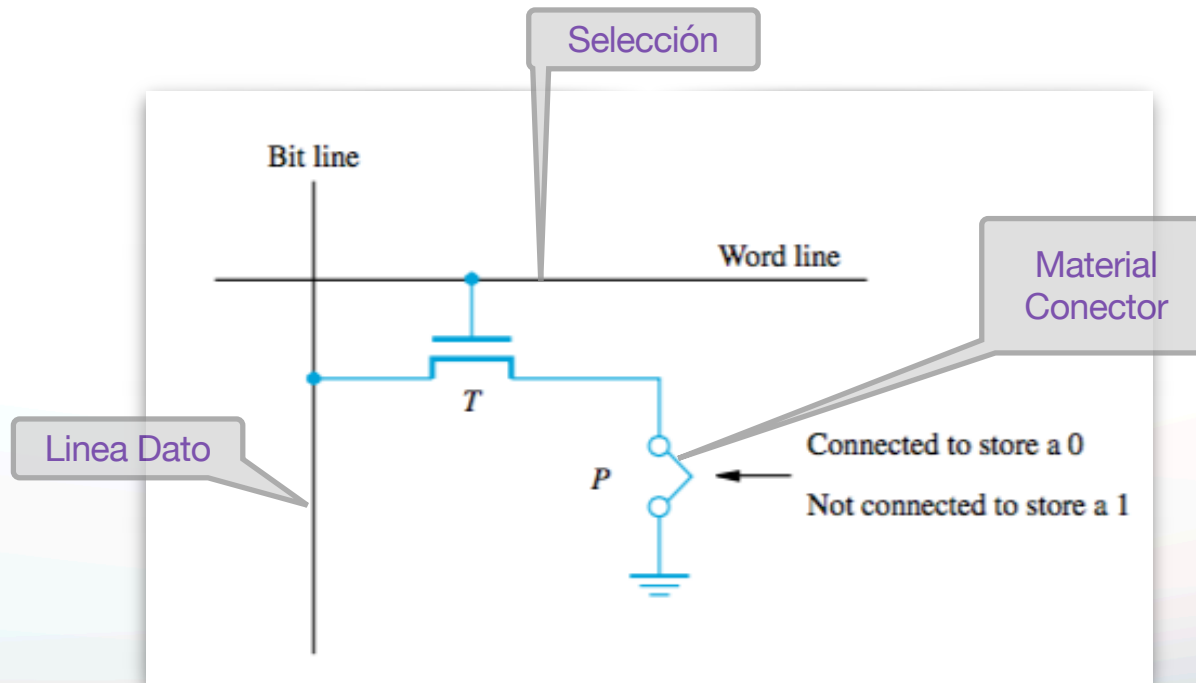
- **Latencia:** Tiempo en obtener el primer Página de datos. Usualmente se mide de manera relativo como cantidad de ciclos de reloj. Por ej. si es de 5 ciclos y el reloj es de 500MHz, entonces el tiempo es de 10 ns.
- **Bandwidth:** Cantidad de datos (bits o Bytes) que la memoria pues transferir por segundo.

Nombre	Reloj E/S	Promedio Transferencia	Bandwith Teórico
DDR-200, PC-1600	100 MHz	0.2 GB/s	1.6 Gb/s
DDR2-800, PC2-6400	400 MHz	0.8 GB/s	6.4 Gb/s
DDR3-1600, PC3-12800	800 MHz	1.6 GB/s	12.8 Gb/s
DDR4-3200, PC4-25600	1600 MHz	3.2 GB/s	25.6 Gb/s
DDR5-6400, PC5-51200	3200 MHz	6.4 GB/s	51.2 Gb/s



Memoria ROM

- **ROM (Read Only Memory)**: Las celdas de memoria no pueden modificarse (durante su utilización).



Tipos memorias ROM

- **PROM** (Programmable ROM): El material conector son fusibles, el proceso de grabado (quema) los de las celdas a contener 1.
- **EPROM** (Erasable Programmable ROM): A diferencia de las PROM, tiene transistores en vez de fusibles, estos puede ser borrados y vueltos a programa mediante luz ultravioleta.
- **EEPROM** (Electrical Erasable Programmable ROM): Son memorias reprogramables con electricidad, a diferencia de las EPROM, utilizan diferentes voltajes para activar o desactivar los transistores de las celdas.
- **FLASH**: En esencia son EEPROM pero que sólo pueden ser modificados de a Páginas reduciendo la complejidad de circuitos y su costo.

Automatizar una sumatoria de n números

0	0	0	1	0	0	0	1	0
0	0	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	0	0	0	1	0	0	1	3
0	0	0	0	0	0	1	1	4
0	0	1	0	1	1	1	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	1	0	0	0	0	0	0	8
0	0	0	0	1	0	1	1	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

¿Cómo implementamos la noción de etapa o paso?, ¿cómo podemos hacer un contador que recorra las direcciones?

A

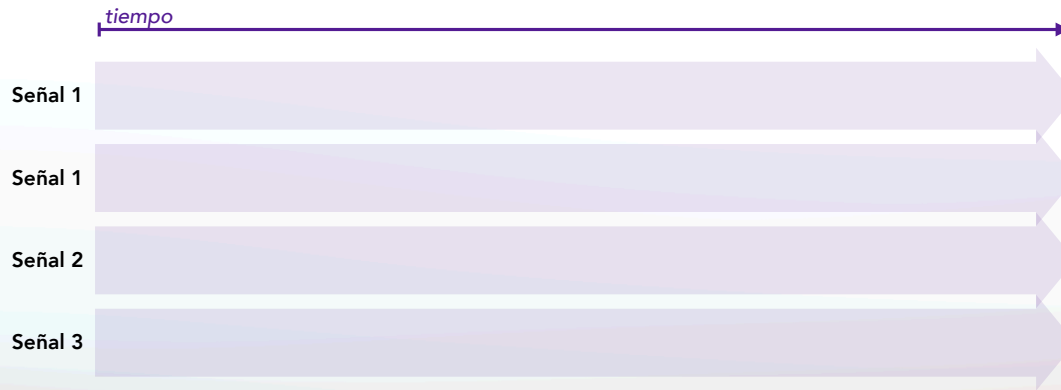
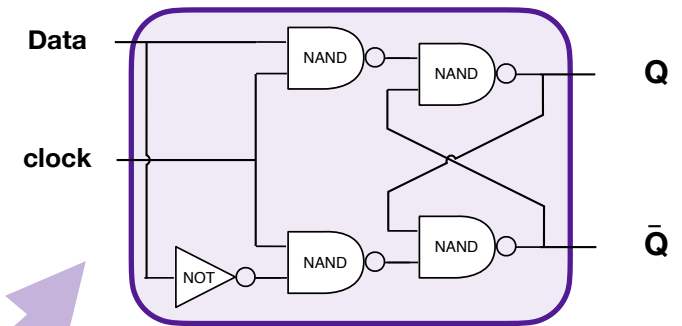
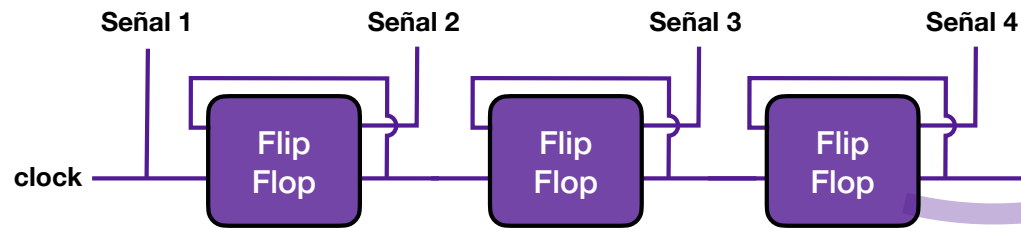
B

Full Adder (n bits)

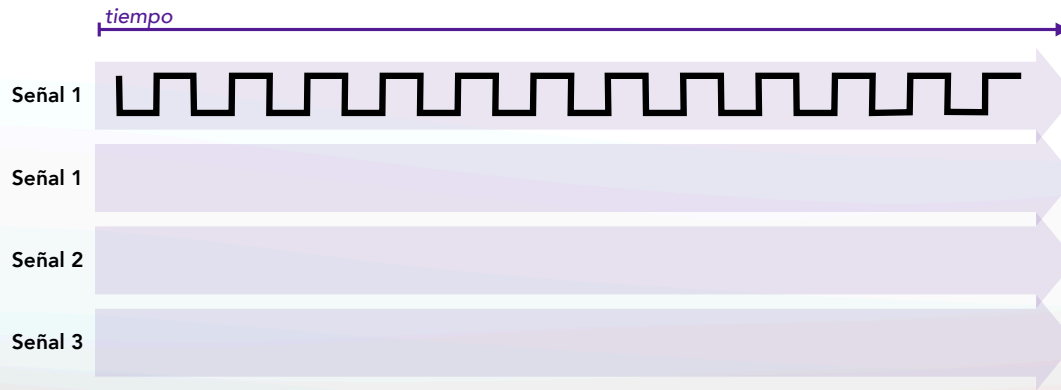
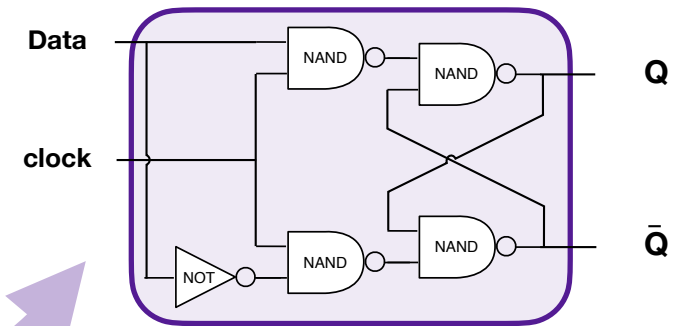
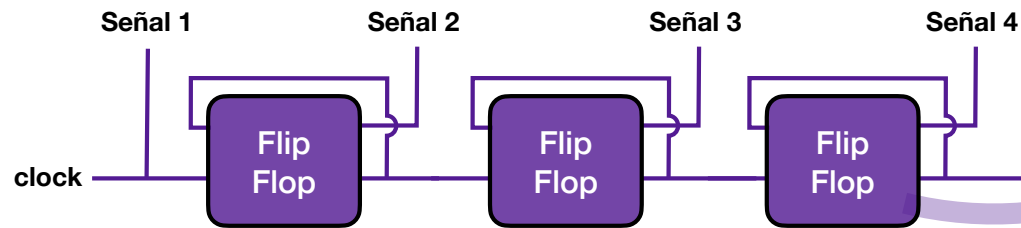
Dirección

Dato

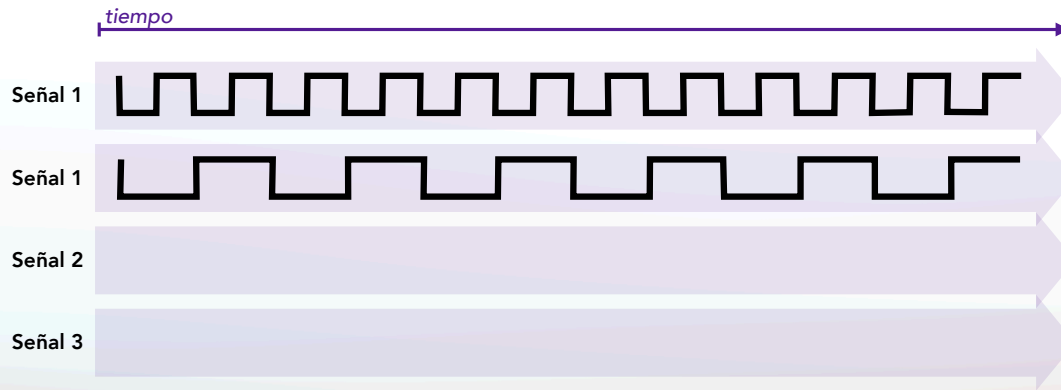
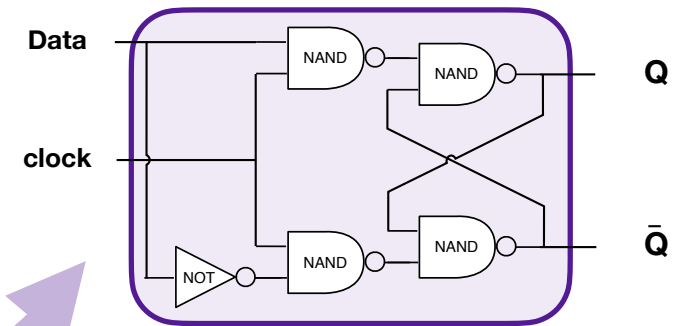
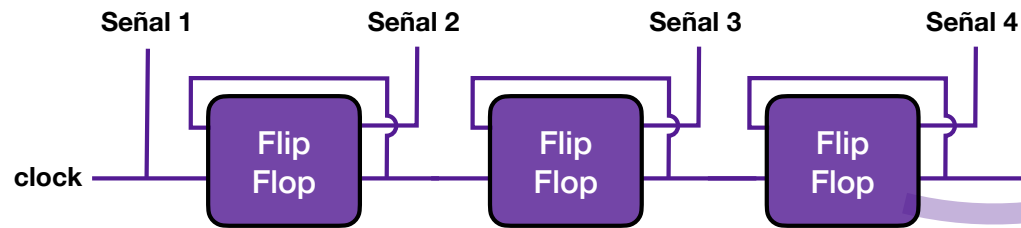
Contador



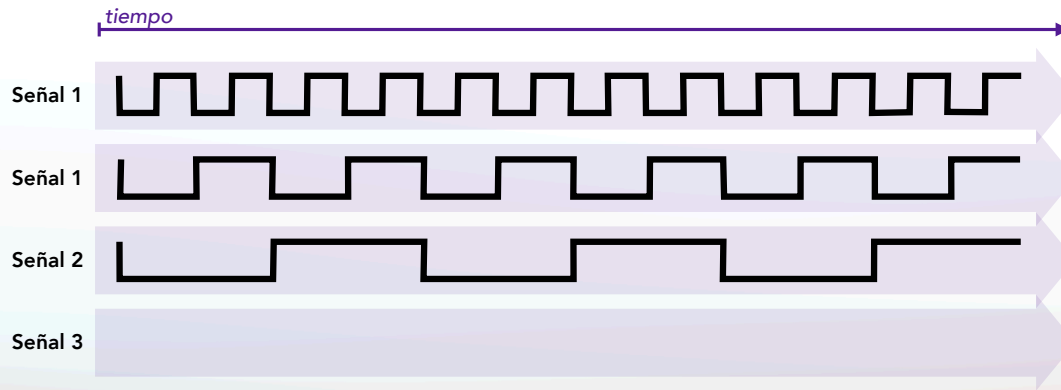
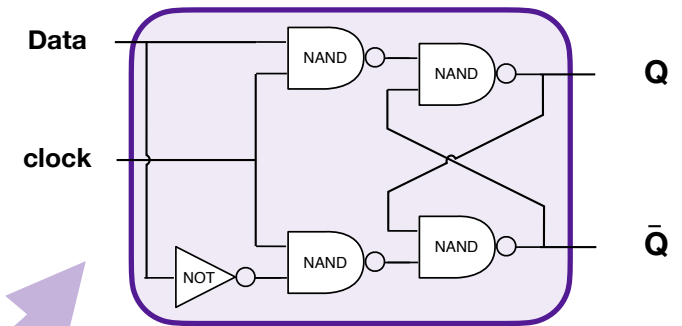
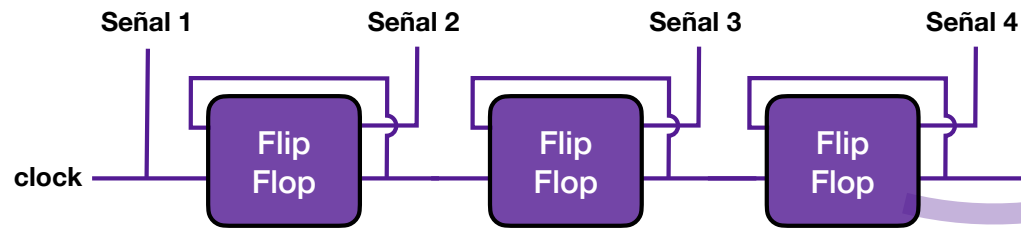
Contador



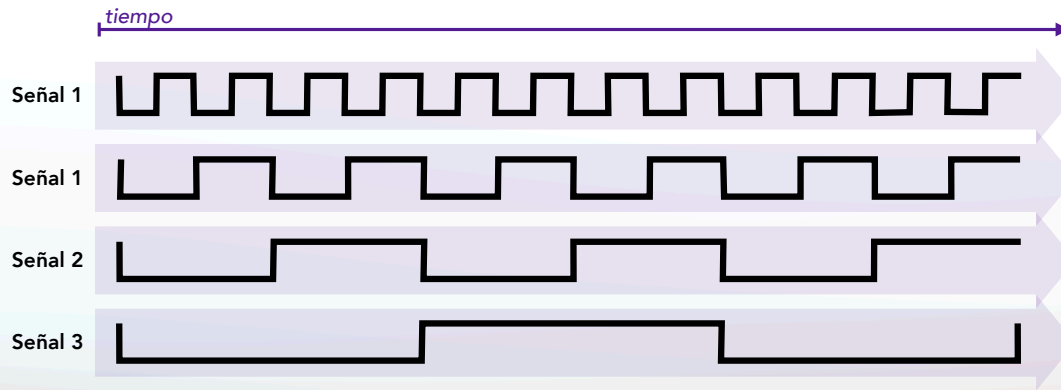
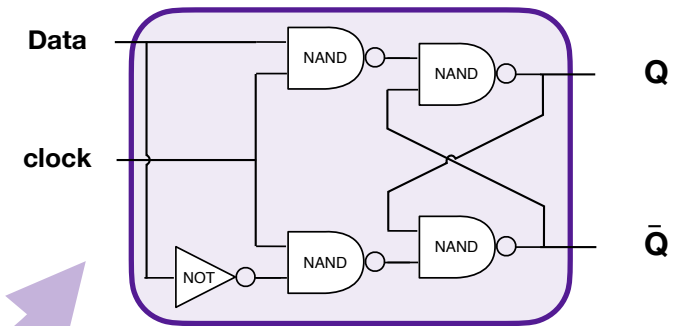
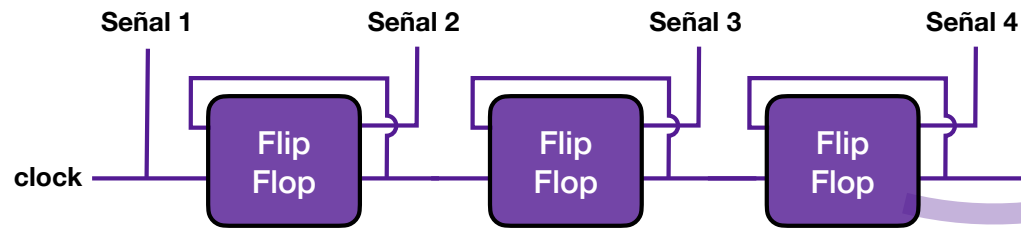
Contador



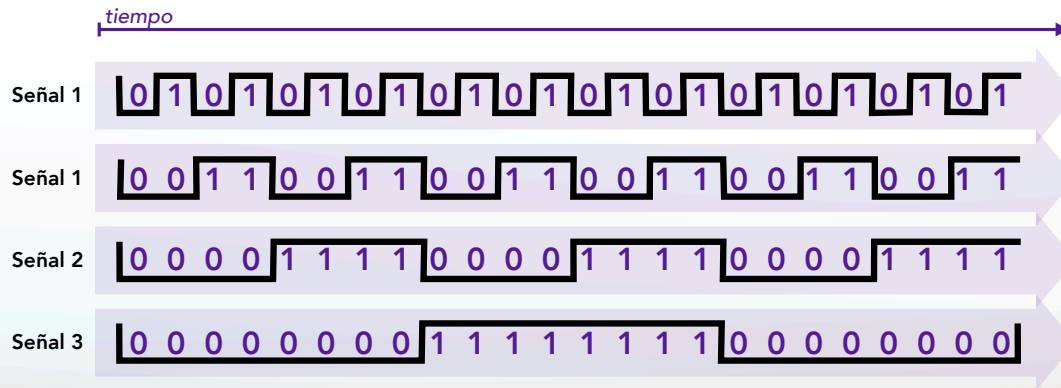
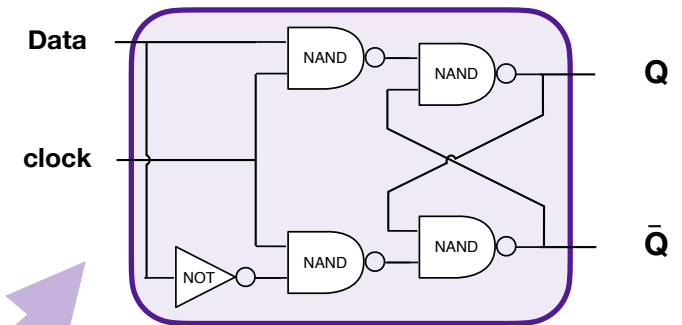
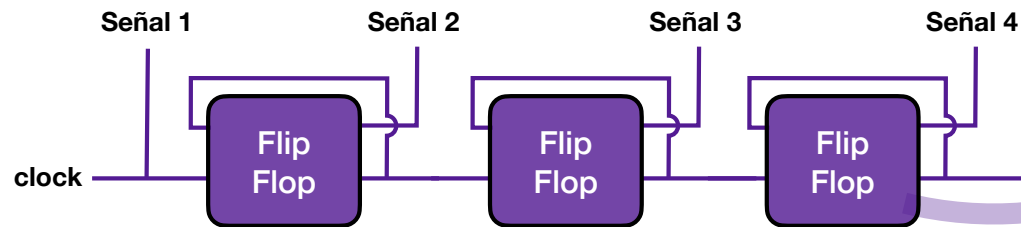
Contador



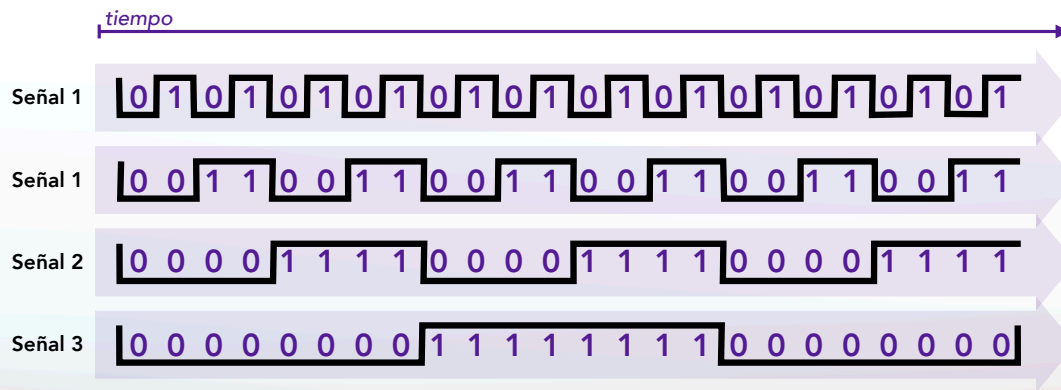
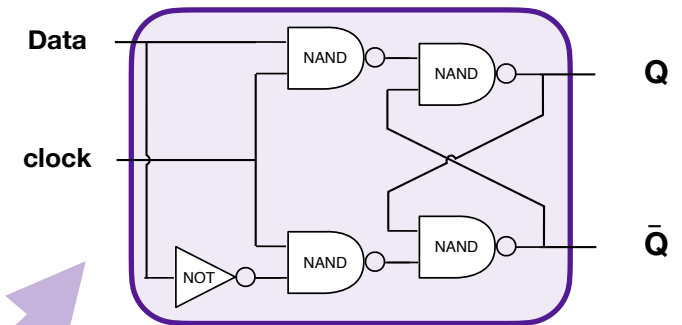
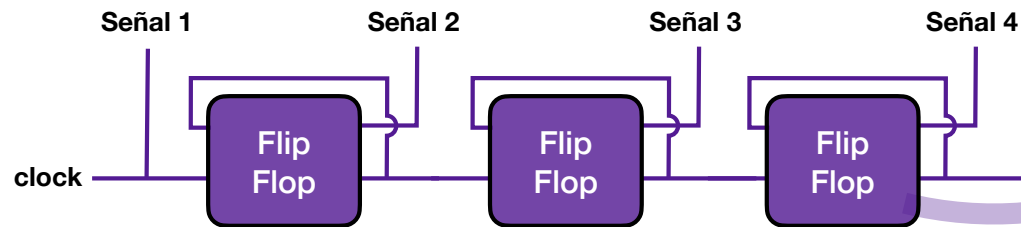
Contador



Contador

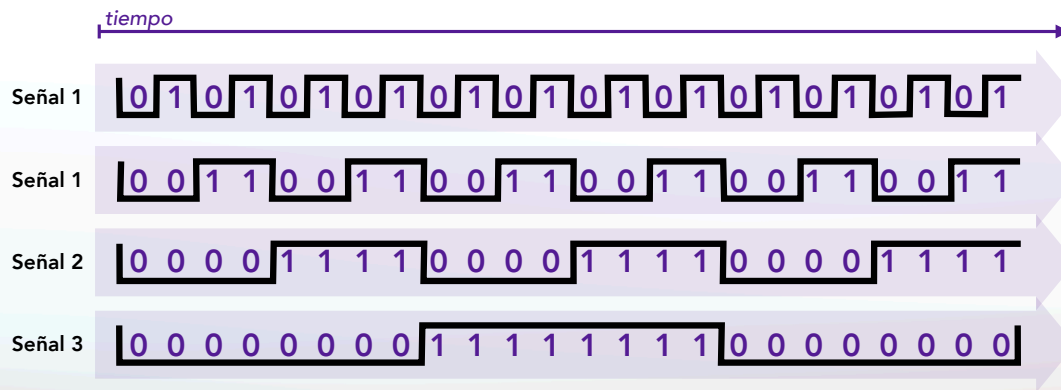
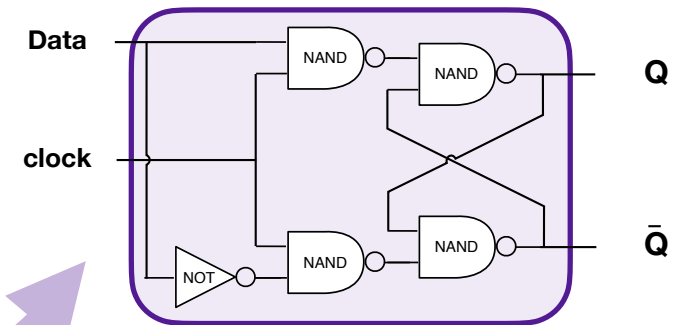
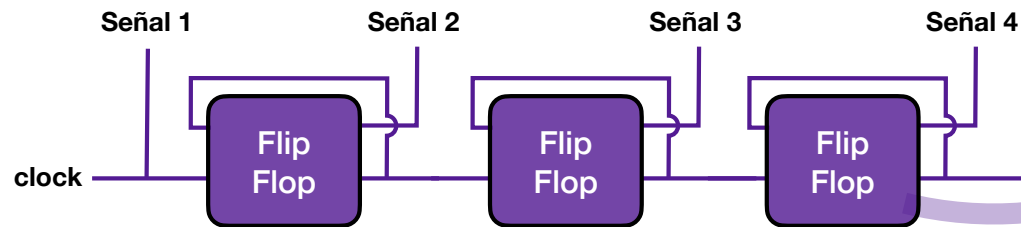


Contador



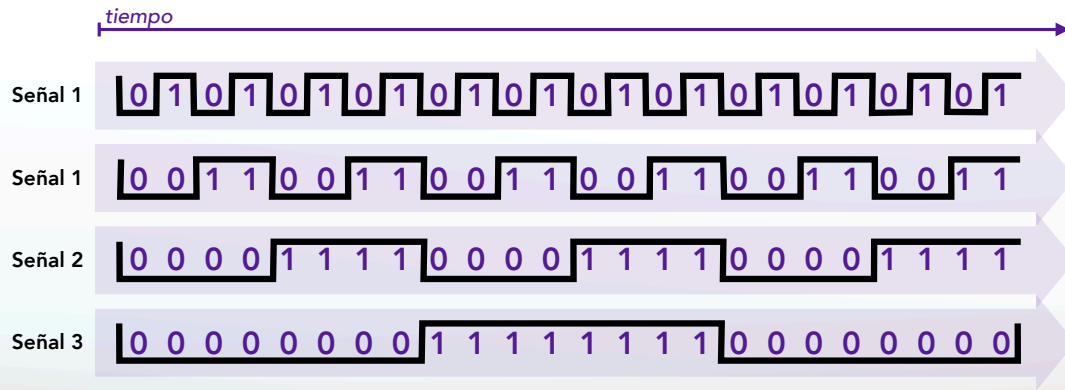
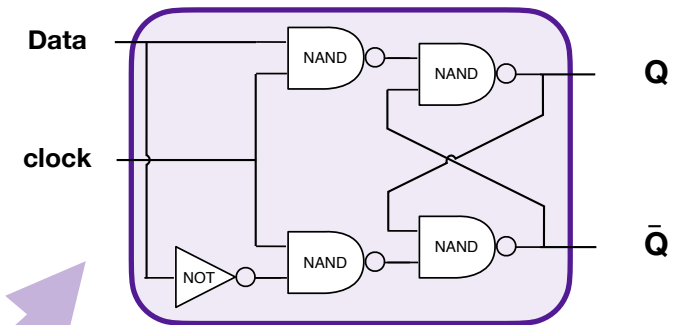
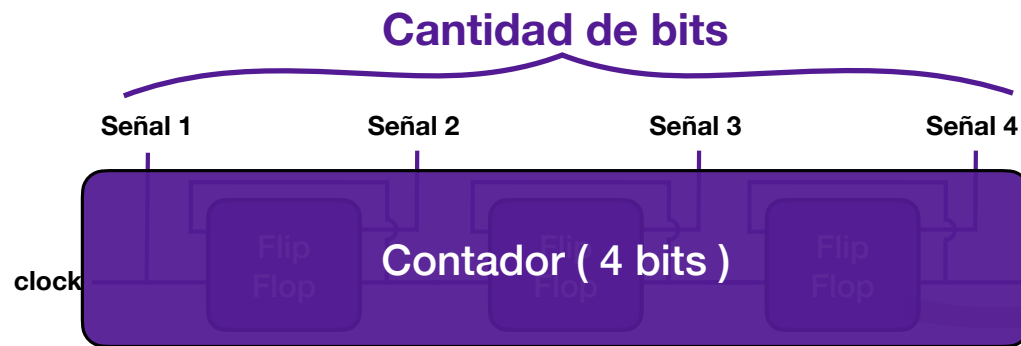
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

Contador



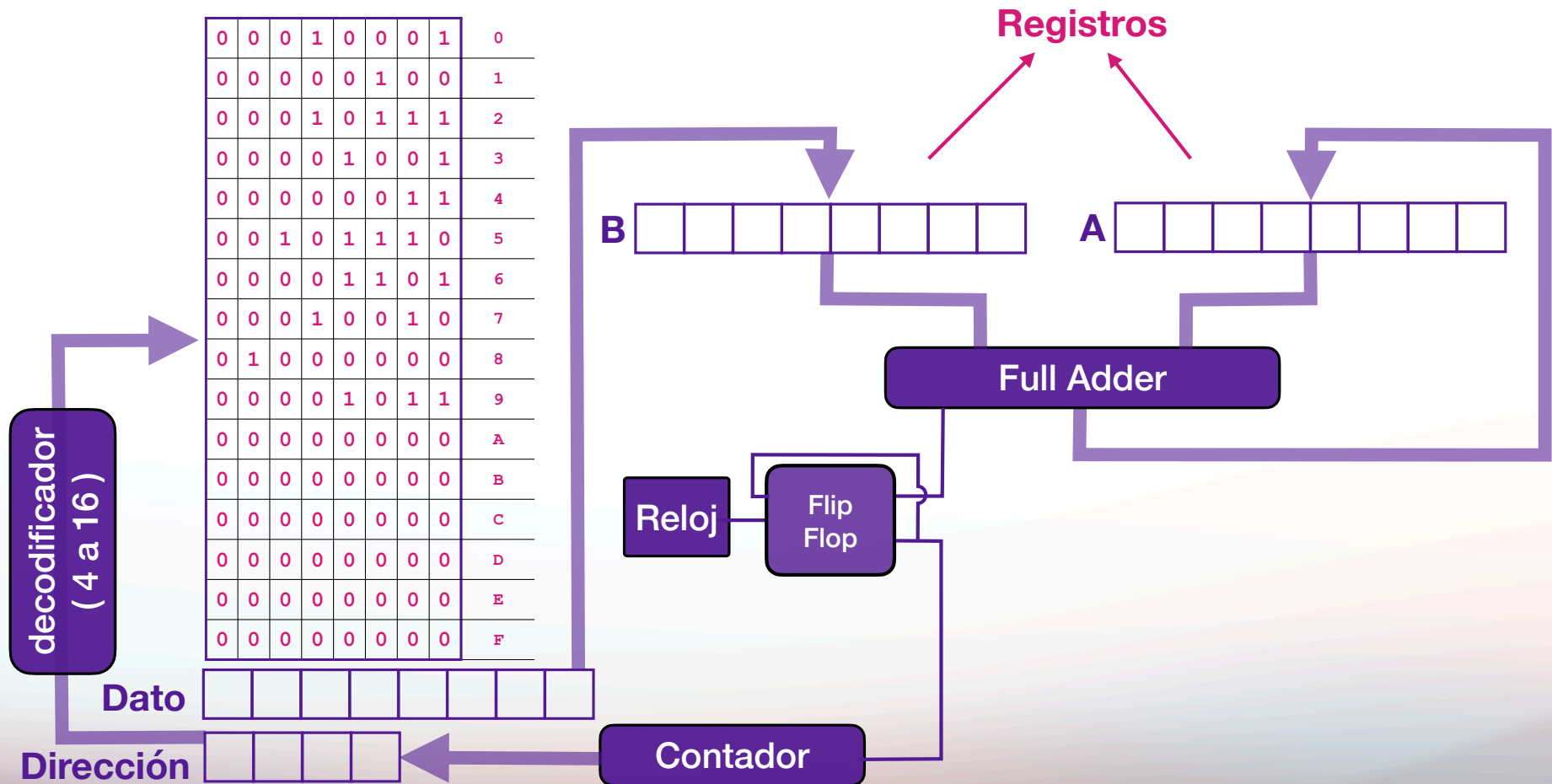
0	0	0	0	→	0
0	0	0	1	→	1
0	0	1	0	→	2
0	0	1	1	→	3
0	1	0	0	→	4
0	1	0	1	→	5
0	1	1	0	→	6
0	1	1	1	→	7
1	0	0	0	→	8
1	0	0	1	→	9
1	0	1	0	→	10
1	0	1	1	→	11
1	1	0	0	→	12
1	1	0	1	→	13
1	1	1	0	→	14
1	1	1	1	→	15

Contador

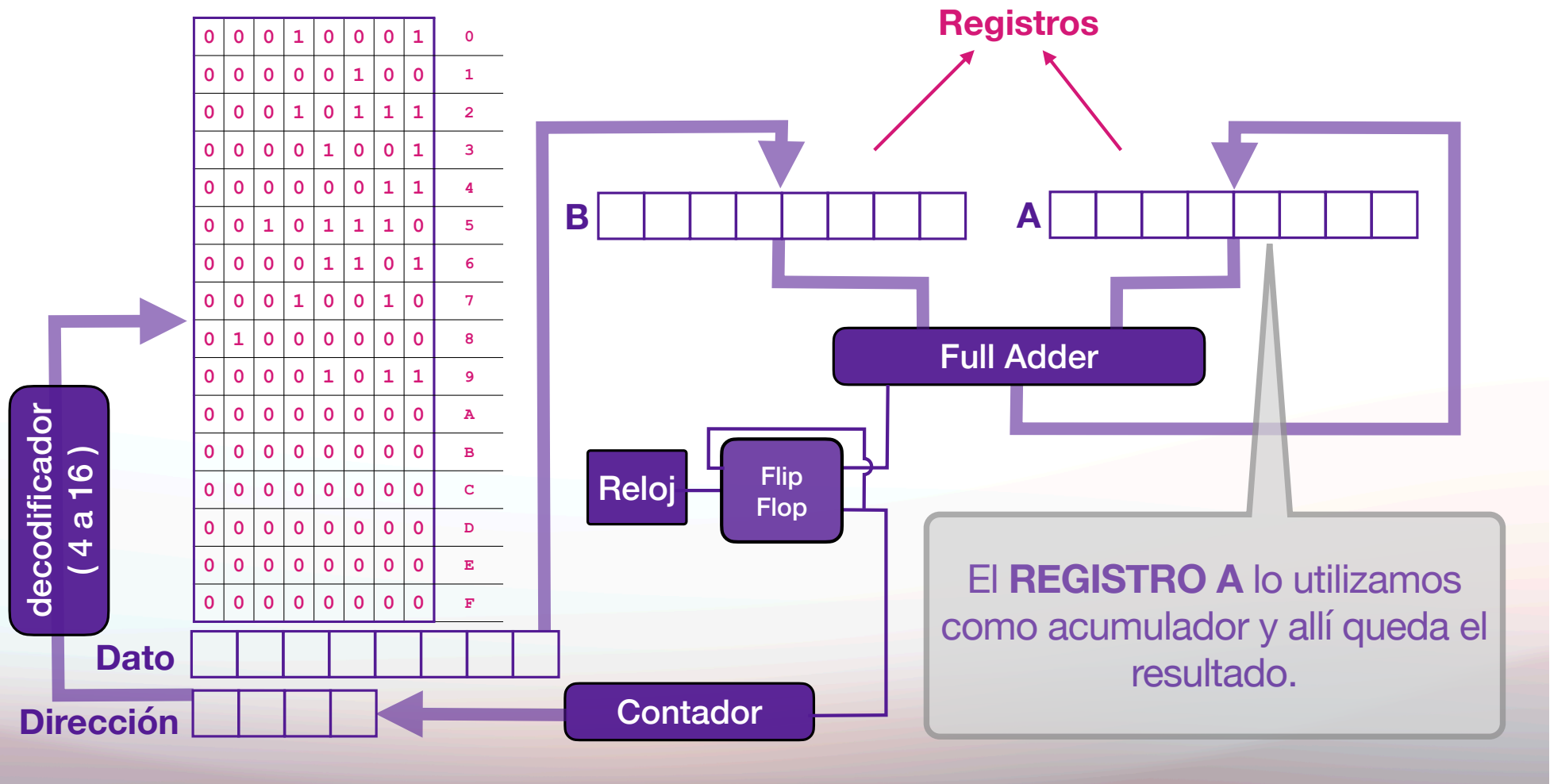


0	0	0	0	→	0
0	0	0	1	→	1
0	0	1	0	→	2
0	0	1	1	→	3
0	1	0	0	→	4
0	1	0	1	→	5
0	1	1	0	→	6
0	1	1	1	→	7
1	0	0	0	→	8
1	0	0	1	→	9
1	0	1	0	→	10
1	0	1	1	→	11
1	1	0	0	→	12
1	1	0	1	→	13
1	1	1	0	→	14
1	1	1	1	→	15

Automatizar una sumatoria de n números

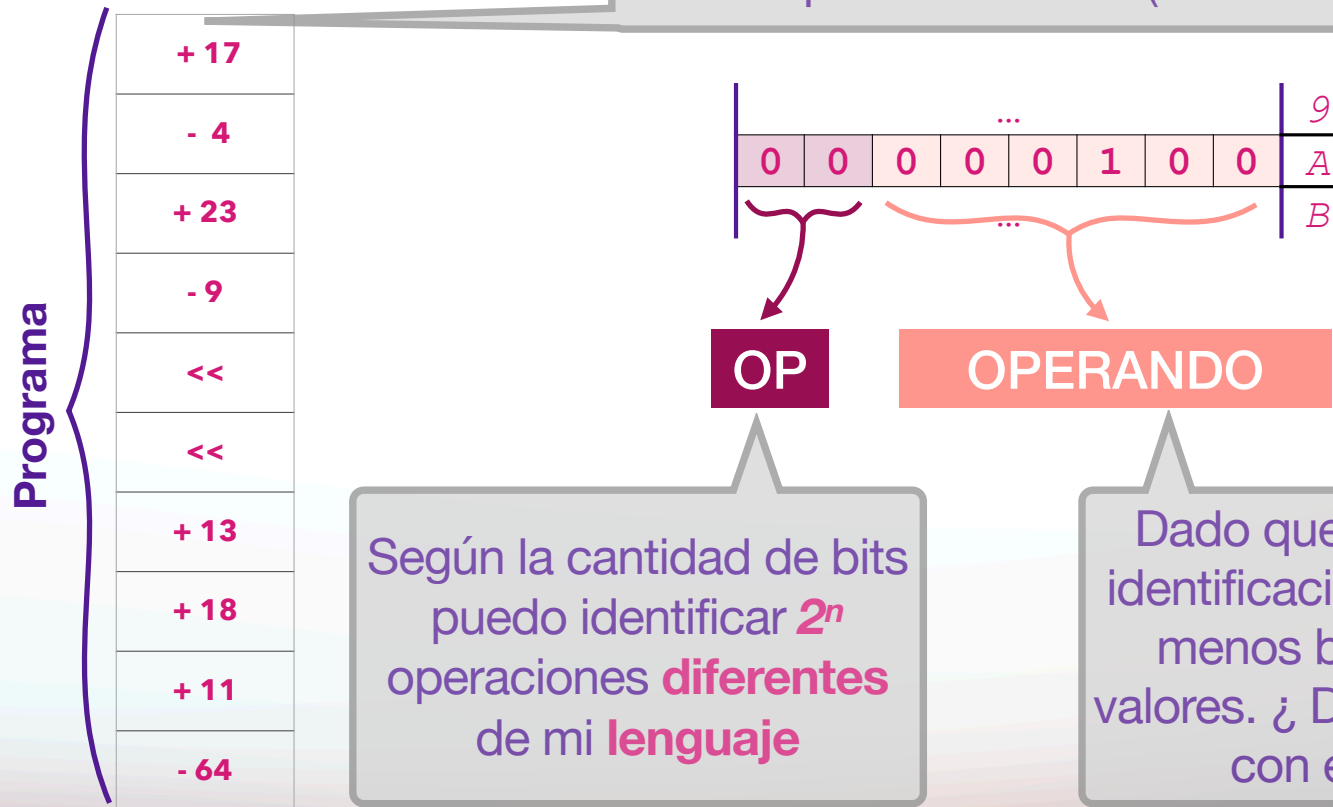


Automatizar una sumatoria de n números



Automatizar una secuencia de operaciones: sumas, restas o *shift*

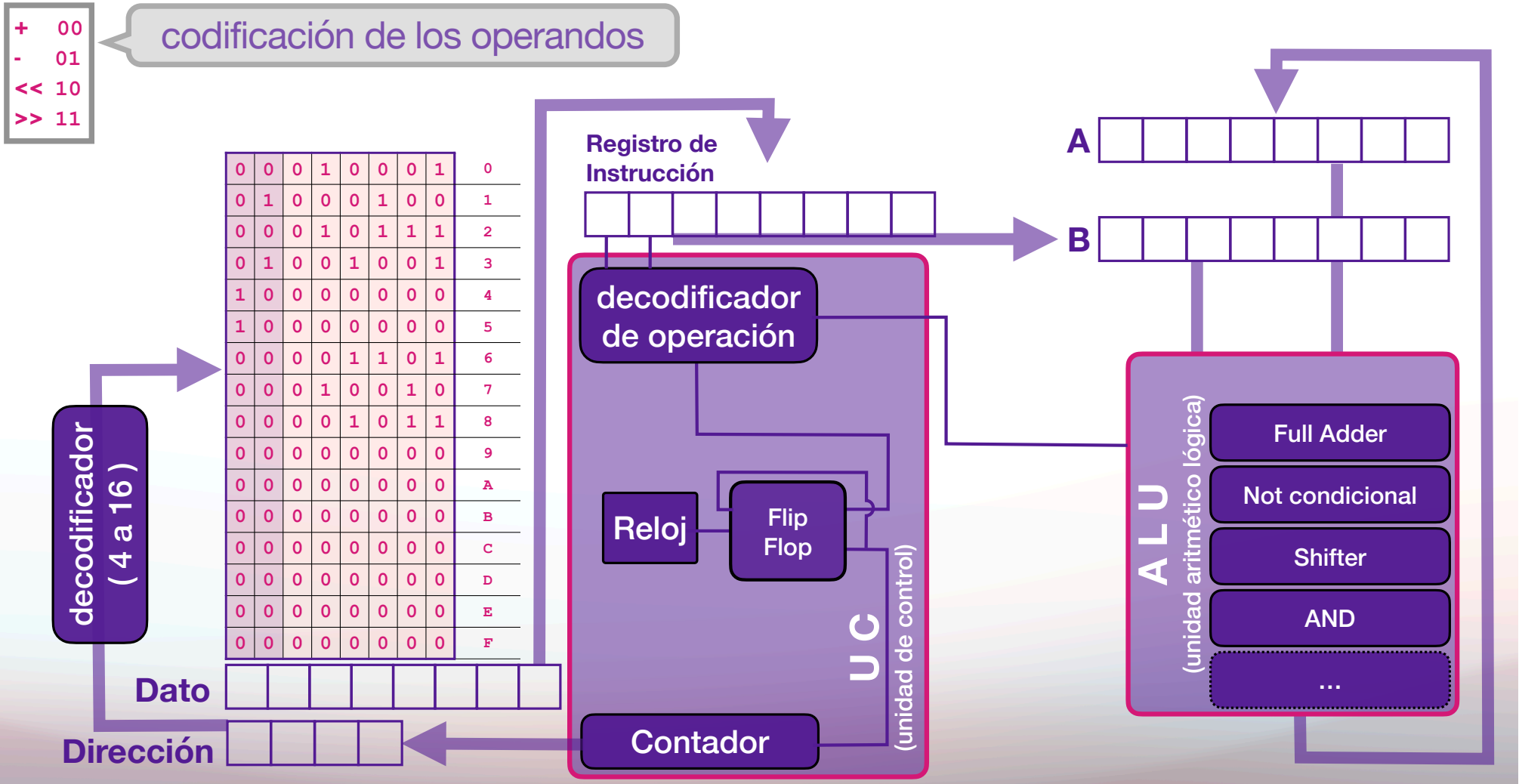
Debemos indicar qué operación y el correspondiente valor (si corresponde)



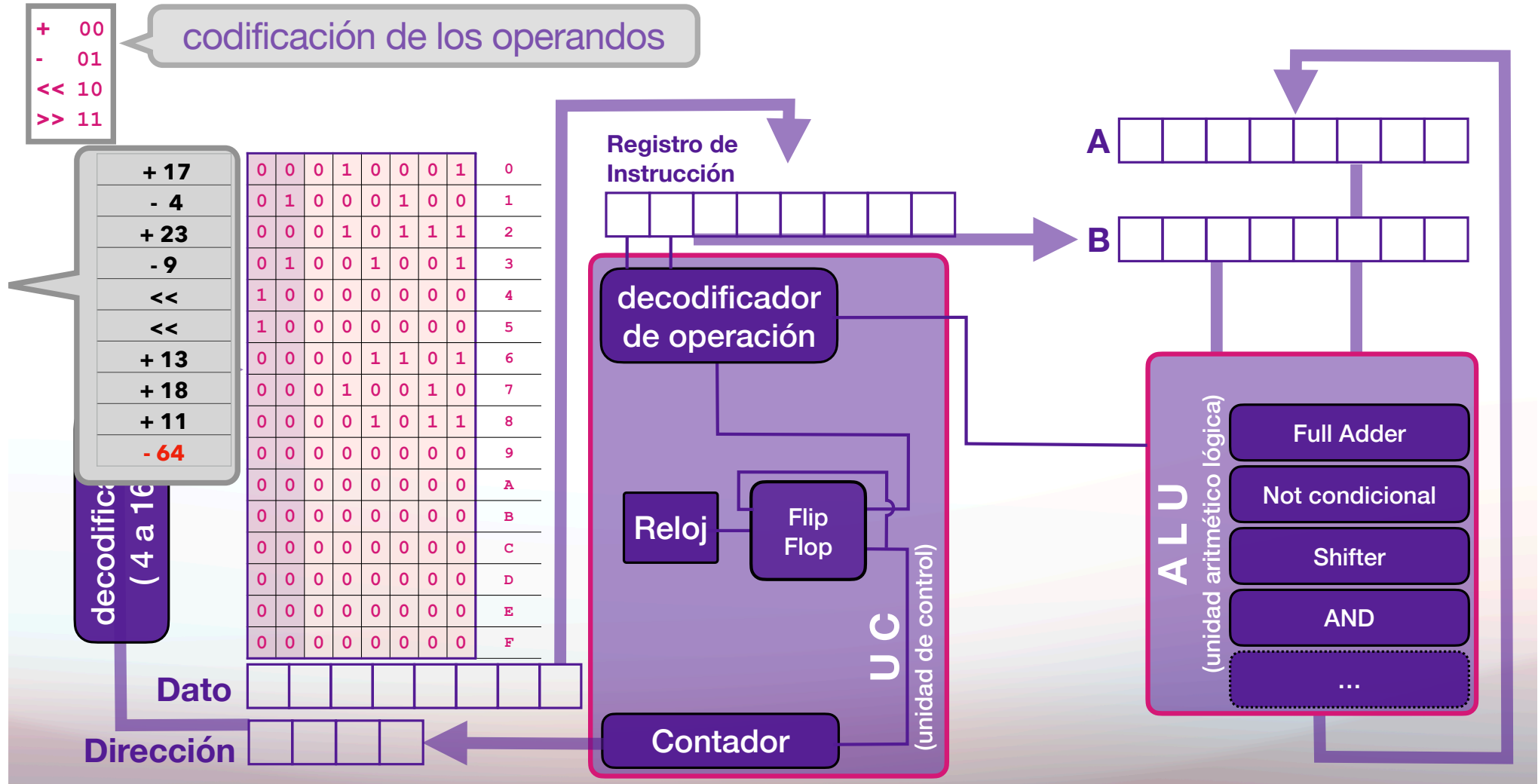
Según la cantidad de bits
puedo identificar 2^n
operaciones **diferentes**
de mi **lenguaje**

Dado que resignamos bits para la
identificación del operador, tenemos
menos bits para representar los
valores. ¿Dejamos algunos de la lista
con esta configuración ?

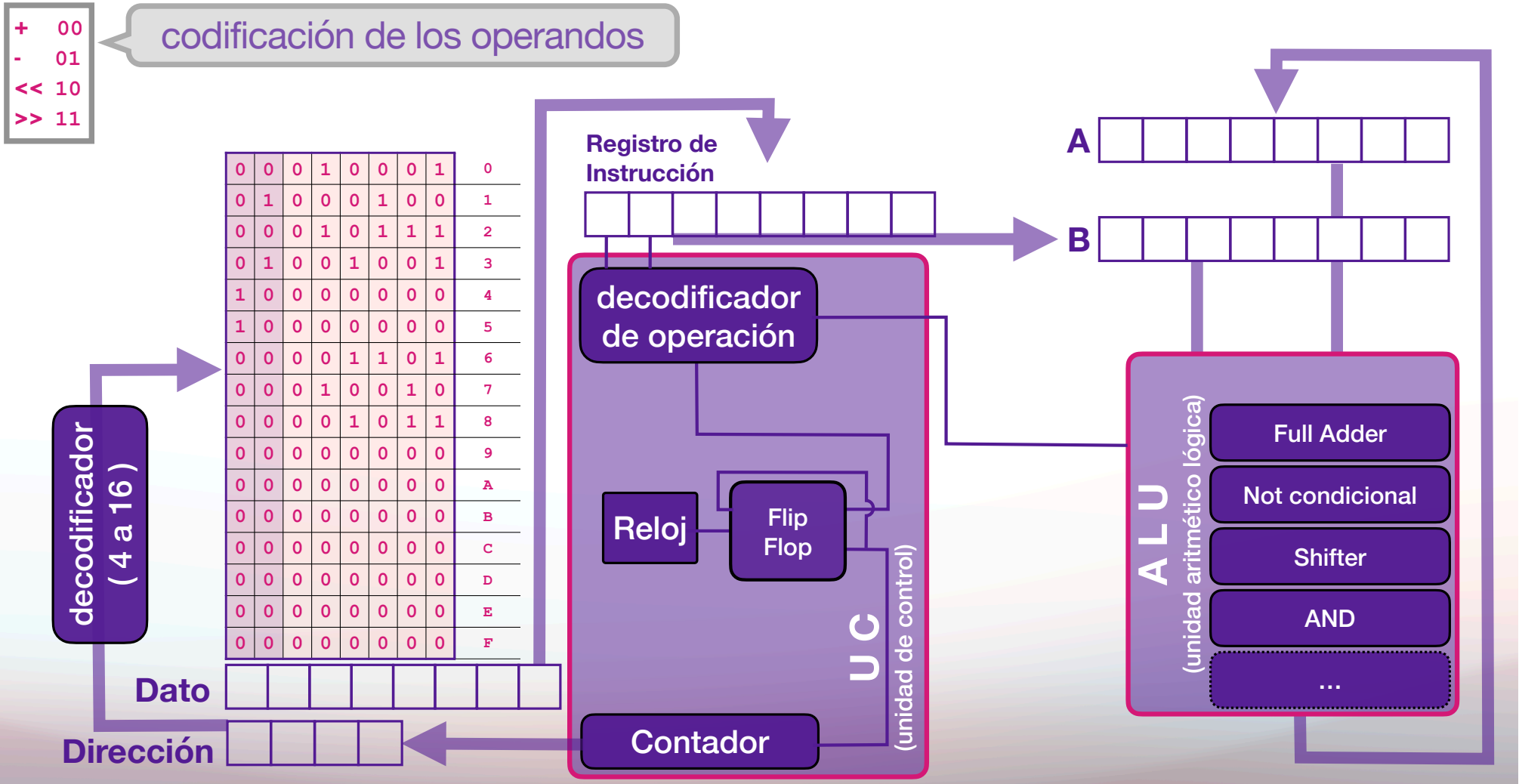
Automatizar una secuencia de operaciones: sumas, restas o *shift*



Automatizar una secuencia de operaciones: sumas, restas o *shift*



Automatizar una secuencia de operaciones: sumas, restas o *shift*



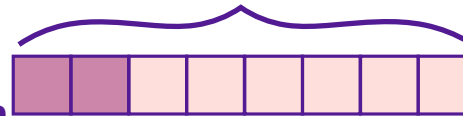
¿Cómo mejorar el espacio de trabajo?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	1	0	0	1	0	0	1	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

8 BITS = tamaño de
arquitectura

Registro
de instrucción



Registros de
uso general



OP

OPERANDO

¿Cómo mejorar el espacio de trabajo?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	1	0	0	1	0	0	1	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

OP

OPERANDO

8 BITS = tamaño de
arquitectura

¿Qué sucede si queremos tener más
operaciones, una mejor representación de
datos o (guardar DATOS en la memoria)?

Teniendo en cuenta que
cada celda tiene **1byte = 8bits**

¿Cómo mejorar el espacio de trabajo?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

16 BITS = tamaño de arquitectura

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	1	0	0	1	0	0	1	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

Registro de instrucción



Teniendo registros de más bits y utilizando 2 celdas de memoria para cada instrucción (o para almacenar datos). Esta arquitectura nos permite configurar, por ej. operaciones con 2 operandos.

Registros de uso general



Little endian vs Big endian

¿Cómo mejorar el espacio de trabajo?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	1	0	0	1	0	0	1	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

Little endian vs Big endian

Registro
de instrucción

16 BITS = tamaño de
arquitectura

INST.	COD	OPERADOR1	OPERADOR2
-------	-----	-----------	-----------

Teniendo registros de más bits y utilizando 2 celdas de memoria para cada instrucción (o para almacenar datos). Esta arquitectura nos permite configurar, por ej. operaciones con 2 operandos.

Registros de
uso general



¿Cómo mejorar el espacio de trabajo?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	1
0	1	0	0	1	0	0	1	1
1	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0
0	0	0	0	1	1	0	1	0
0	0	0	1	0	0	1	0	0
0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Registro
de instrucción

INST. COD OPERADOR1 OPERADOR2

16 BITS = tamaño de
arquitectura

Teniendo registros de más bits y utilizando 2 celdas de memoria para cada instrucción (o para almacenar datos). Esta arquitectura nos permite configurar, por ej. operaciones con 2 operandos.

Para usar la memoria para guardar DATOS, puedo acceder a ellos a través de su dirección de memoria, es decir, el valor del operando es la dirección donde está almacenada la información

Registros de
uso general



Little endian vs Big endian

¿Cómo leer o guardar valores en la memoria?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
1	0	1	0	0	0	1	0	0
2	0	0	0	1	0	1	1	1
3	0	1	0	0	1	0	0	1
4	1	0	0	0	0	0	0	0
5	1	0	0	0	0	0	0	0
6	0	0	0	0	1	1	0	1
7	0	0	0	1	0	0	1	0
8	0	0	0	0	1	0	1	1
9	0	0	0	0	0	0	0	0
A	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0
C	0	0	0	0	0	0	0	0
D	0	0	0	0	0	0	0	0
E	0	0	0	0	0	0	0	0
F	0	0	0	0	0	0	0	0

INST. COD OPERADOR1 OPERADOR2

Ejemplo

GUARDAR DIRECCIÓN C REGISTRO A

0 1 1 1 0 0 1 1 0 0 0 0 0 0 0 0

Registro de instrucción

...
Guardar 07 (0111)
...

...
Dirección C (001100)
...

...
RegistroA (000000)
...

0 0 0 0 0 0 0 1 0 1 0 1 0 1 1 1

Registros de uso general A

--	--	--	--	--	--

Dirección

--

Leer/Escribir

¿Cómo leer o guardar valores en la memoria?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
1	0	1	0	0	0	1	0	0
2	0	0	0	1	0	1	1	1
3	0	1	0	0	1	0	0	1
4	1	0	0	0	0	0	0	0
5	1	0	0	0	0	0	0	0
6	0	0	0	0	1	1	0	1
7	0	0	0	1	0	0	1	0
8	0	0	0	0	1	0	1	1
9	0	0	0	0	0	0	0	0
A	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0
C	0	0	0	0	0	0	0	0
D	0	0	0	0	0	0	0	0
E	0	0	0	0	0	0	0	0
F	0	0	0	0	0	0	0	0

--	--	--	--	--	--

Dirección

1

Leer/Escribir

INST. COD OPERADOR1 OPERADOR2

Ejemplo

GUARDAR DIRECCIÓN C REGISTRO A

0 1 1 1 0 0 1 1 0 0 0 0 0 0 0 0

Registro de instrucción

...
Guardar 07 (0111)
...

...
Dirección C (001100)
...

...
RegistroA (000000)
...

0 0 0 0 0 0 0 1 0 1 0 1 0 1 1 1

Registros de uso general A

¿Cómo leer o guardar valores en la memoria?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
1	0	1	0	0	0	1	0	0
2	0	0	0	1	0	1	1	1
3	0	1	0	0	1	0	0	1
4	1	0	0	0	0	0	0	0
5	1	0	0	0	0	0	0	0
6	0	0	0	0	1	1	0	1
7	0	0	0	1	0	0	1	0
8	0	0	0	0	1	0	1	1
9	0	0	0	0	0	0	0	0
A	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0
C	0	0	0	0	0	0	0	0
D	0	0	0	0	0	0	0	0
E	0	0	0	0	0	0	0	0
F	0	0	0	0	0	0	0	0

0 0 1 1 0 0

Dirección

1

Leer/Escribir

INST. COD OPERADOR1 OPERADOR2

Ejemplo

GUARDAR DIRECCIÓN C REGISTRO A

0 1 1 1 0 0 1 1 0 0 0 0 0 0 0 0

Registro de instrucción

...
Guardar 07 (0111)
...

...
Dirección C (001100)
...

...
RegistroA (000000)
...

0 0 0 0 0 0 0 1 0 1 0 1 0 1 1 1

Registros de uso general A

¿Cómo leer o guardar valores en la memoria?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
1	0	1	0	0	0	1	0	0
2	0	0	0	1	0	1	1	1
3	0	1	0	0	1	0	0	1
4	1	0	0	0	0	0	0	0
5	1	0	0	0	0	0	0	0
6	0	0	0	0	1	1	0	1
7	0	0	0	1	0	0	1	0
8	0	0	0	0	1	0	1	1
9	0	0	0	0	0	0	0	0
A	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0
C	0	0	0	0	0	0	0	1
D	0	1	0	1	0	1	1	1
E	0	0	0	0	0	0	0	0
F	0	0	0	0	0	0	0	0

0 0 1 1 0 0

Dirección

1

Leer/Escribir

INST. COD OPERADOR1 OPERADOR2

Ejemplo

GUARDAR DIRECCIÓN C REGISTRO A

0 1 1 1 0 0 1 1 0 0 0 0 0 0 0 0

Registro de instrucción

...
Guardar 07 (0111)
...

...
Dirección C (001100)
...

...
RegistroA (000000)
...

0 0 0 0 0 0 0 1 0 1 0 1 0 1 1 1

Registros de uso general A

¿Cómo podemos controlar el orden de ejecución de instrucciones?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	1	0	0	1	0	0	1	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

Program Counter
(Contador de Programa)

--	--	--	--	--	--

El Program Counter contiene la dirección de la próxima instrucción a ejecutar

Registro de
Instrucción

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

A

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

B

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

U C

(unidad de control)

A L U

(unidad aritmético lógica)

--	--	--	--	--	--

Dirección

0

Leer/Escribir

¿Cómo podemos controlar el orden de ejecución de instrucciones?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	1	0	0	1	0	0	1	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

Program Counter
(Contador de Programa)

0 0 0 0 0 0

El Program Counter contiene la dirección de la próxima instrucción a ejecutar

Registro de
Instrucción

A

B

UC

(unidad de control)

ALU

(unidad aritmético lógica)

0 0 0 0 0 0

Dirección

0

Leer/Escribir

¿Cómo podemos controlar el orden de ejecución de instrucciones?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	1	0	0	1	0	0	1	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

Program Counter
(Contador de Programa)

0 0 0 0 0 0

Registro de
Instrucción

0 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0

A

16 empty boxes for register A

B

16 empty boxes for register B

UC

(unidad de control)

ALU

(unidad aritmético lógica)

0 0 0 0 0 0

Dirección

0

Leer/Escribir

El Program Counter contiene la dirección de la próxima instrucción a ejecutar

¿Cómo podemos controlar el orden de ejecución de instrucciones?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	1	0	0	1	0	0	1	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

Program Counter
(Contador de Programa)

--	--	--	--	--	--

+2

Registro de
Instrucción

0	0	0	1	0	0	0	1	0	1	0	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

A

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

B

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

U C

(unidad de control)

A L U

(unidad aritmético lógica)

--	--	--	--	--	--

Dirección

0

Leer/Escribir

El Program Counter contiene la **dirección** de la próxima instrucción a ejecutar

Luego de traer una instrucción (por defecto) lo **incremento** para apuntar a la siguiente

¿Cómo podemos controlar el orden de ejecución de instrucciones?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	1	0	0	1	0	0	1	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

Program Counter
(Contador de Programa)

0 0 0 0 1 0 +2

Registro de
Instrucción

0 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0

A

B

UC

(unidad de control)

ALU

(unidad aritmético lógica)

0 0 0 0 1 0

Dirección

0

Leer/Escribir

El Program Counter contiene la **dirección** de la próxima instrucción a ejecutar

Luego de traer una instrucción (por defecto) lo **incremento** para apuntar a la siguiente

¿Cómo podemos controlar el orden de ejecución de instrucciones?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	0	0	1	0	1	1	1	2
0	1	0	0	1	0	0	1	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

Program Counter
(Contador de Programa)

0 0 0 0 1 0 +2

Registro de
Instrucción

0 0 0 1 0 1 1 1 0 1 0 0 1 0 0 1

A

B

UC

(unidad de control)

ALU

(unidad aritmético lógica)

0 0 0 0 1 0

Dirección

0

Leer/Escribir

El Program Counter contiene la **dirección** de la próxima instrucción a ejecutar

Luego de traer una instrucción (por defecto) lo **incremento** para apuntar a la siguiente

¿Cómo podemos *alterar* el orden secuencial?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	1	0	0	0	0	0	0	2
0	0	0	0	1	0	0	0	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

Program Counter
(Contador de Programa)

--	--	--	--	--	--

Registro de
Instrucción

0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Podemos tener una **instrucción de salto** que indique cuál es la próxima instrucción a ejecutar

+	0000
-	0001
<<	0010
>>	0011
JMP	0100

UC

(unidad de control)

ALU

(unidad aritmético lógica)

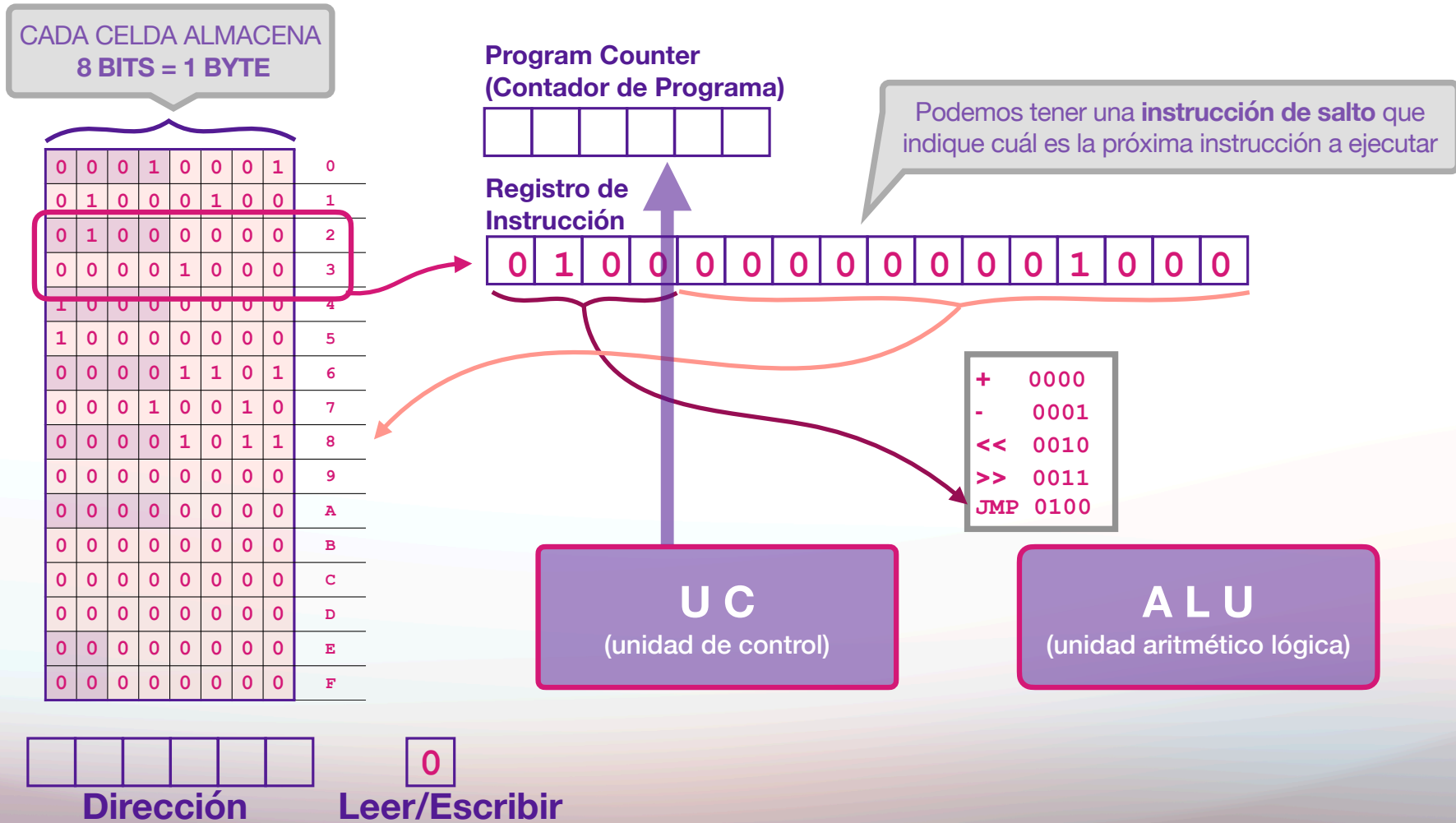
--	--	--	--	--	--

Dirección

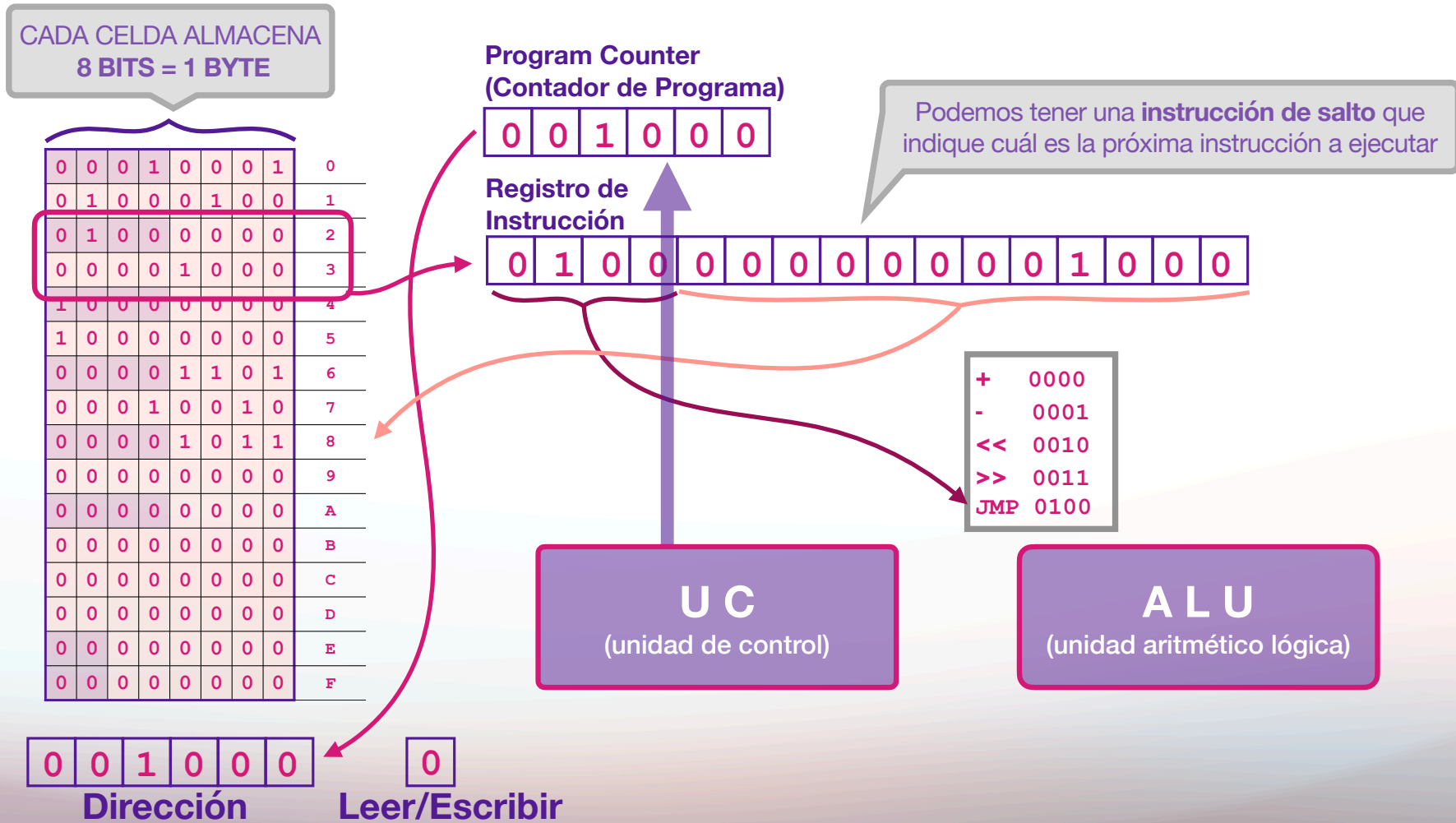
0

Leer/Escribir

¿Cómo podemos *alterar* el orden secuencial?



¿Cómo podemos *alterar* el orden secuencial?



¿Cómo podemos *alterar* el orden secuencial?

CADA CELDA ALMACENA
8 BITS = 1 BYTE

0	0	0	1	0	0	0	1	0
0	1	0	0	0	1	0	0	1
0	1	0	0	0	0	0	0	2
0	0	0	0	1	0	0	0	3
1	0	0	0	0	0	0	0	4
1	0	0	0	0	0	0	0	5
0	0	0	0	1	1	0	1	6
0	0	0	1	0	0	1	0	7
0	0	0	0	1	0	1	1	8
0	0	0	0	0	0	0	0	9
0	0	0	0	0	0	0	0	A
0	0	0	0	0	0	0	0	B
0	0	0	0	0	0	0	0	C
0	0	0	0	0	0	0	0	D
0	0	0	0	0	0	0	0	E
0	0	0	0	0	0	0	0	F

Program Counter
(Contador de Programa)

0 0 1 0 1 0 +2

Registro de
Instrucción

0 0 0 0 1 0 1 1 0 0 0 0 0 0 0 0

Podemos tener una **instrucción de salto** que indique cuál es la próxima instrucción a ejecutar

+ 0000
- 0001
<< 0010
>> 0011
JMP 0100

UC

(unidad de control)

ALU

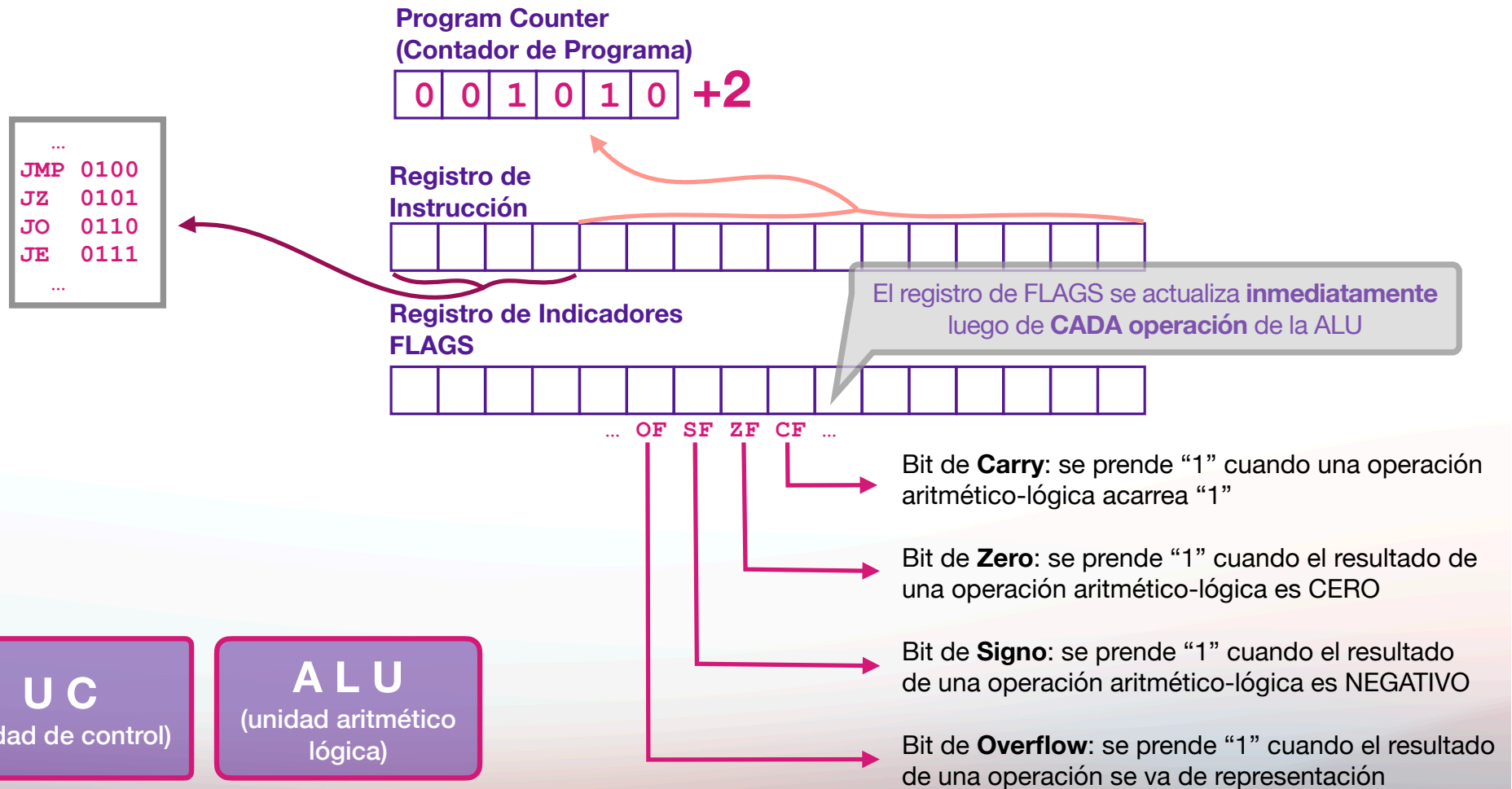
(unidad aritmético lógica)

Dirección

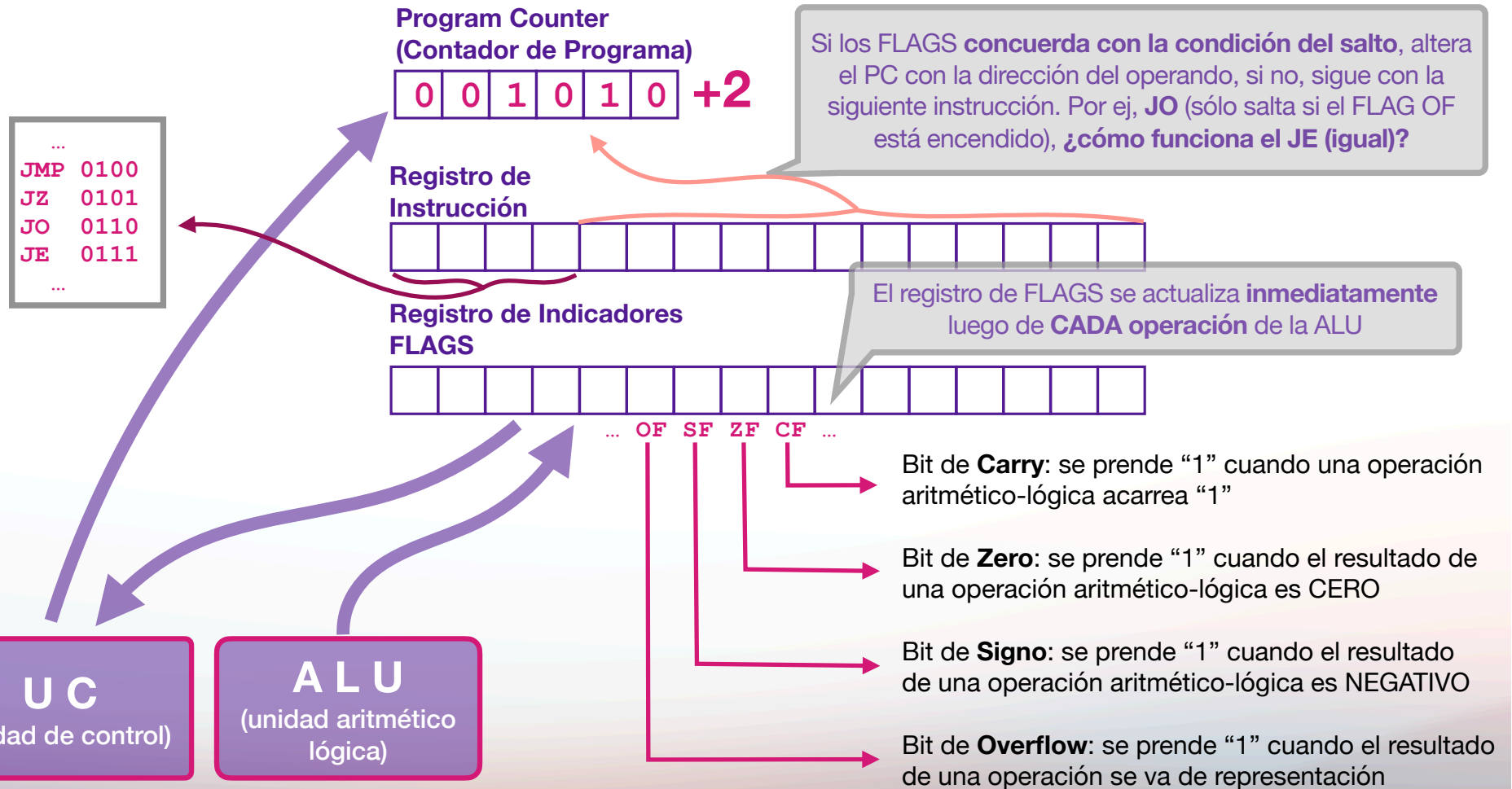
0

Leer/Escribir

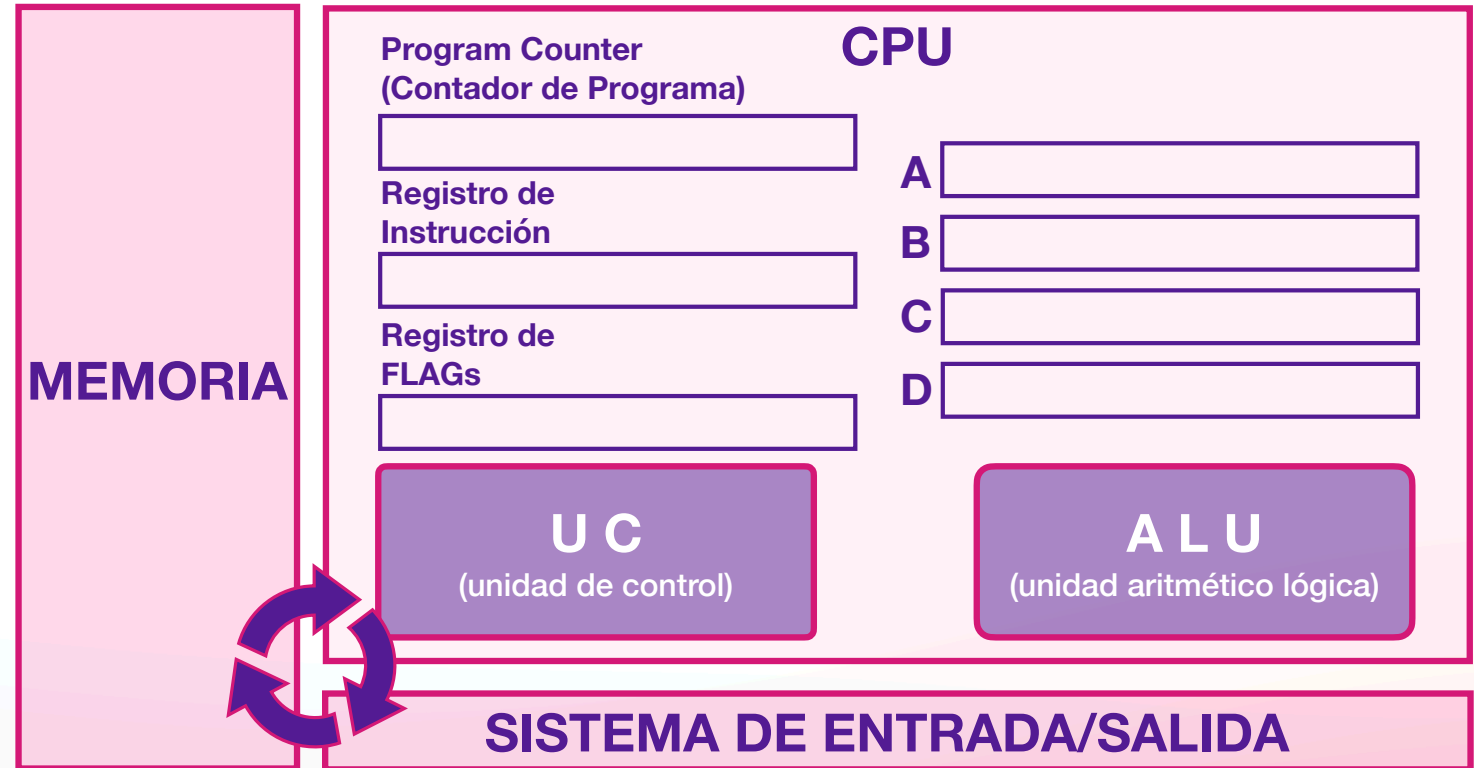
¿Cómo podemos *alterar* el orden secuencial bajo **CONDICIONES**?



¿Cómo podemos *alterar* el orden secuencial bajo **CONDICIONES**?



Arquitectura von Neumann

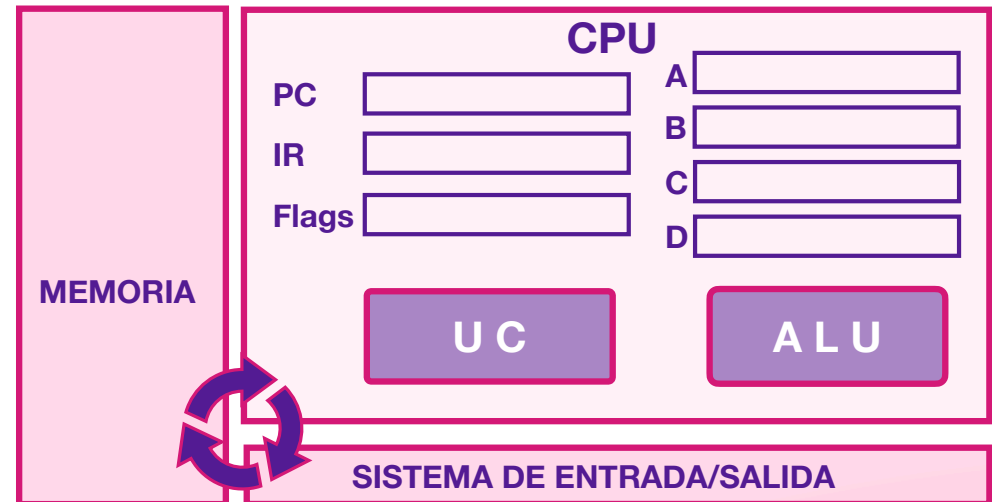


John von Neumann: (1903-1957) Físico Matemático Húngaro, Estadounidense. Participó activamente en el proyecto ENIAC y en el desarrollo de la EDVAC, donde propuso el concepto de almacenamiento de programas y datos en la memoria de la computadora, sentando las bases de la arquitectura de von Neumann, que se convirtió en el modelo predominante en las computadoras modernas.

Arquitectura von Neumann

Fetch: La Unidad de control obtiene de la memoria la próxima instrucción que indica el *contador de programa* (**PC**) y la almacena en el Registro de Instrucción (**IR**). Finalmente actualiza el **PC** indicando la dirección de la próxima instrucción.

Decode: La Unidad de control decodifica la instrucción y obtiene de la memoria (si fuere necesario) la información que involucra dicha instrucción.



Execute: La ALU ejecuta (calcula) el resultado de la operación y lo almacena en un registro o memoria.